



École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes
- Rabat

Projet de Fin d'Année

Platforme intelligente d'indexation et de recherche multimodale avec un moteur RAG

Élèves :
Hajar Braidi

Jury :
Prénom Nom
Prénom Nom

Année Universitaire 2026-2027



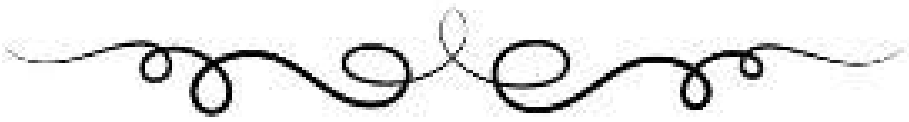
Remerciements

Au terme de ce stage, je tiens tout d'abord à exprimer ma sincère gratitude à **Nabil Nait Serhane**, mon encadrant au sein du département Radio, service Archivage de la Société Nationale de Radiodiffusion et de Télévision, pour sa confiance, sa disponibilité et ses orientations précieuses tout au long de la réalisation de ce projet.

Je remercie également **Taoufik Rachad**, mon encadrant académique, pour son suivi et ses conseils qui m'ont permis de mener ce travail dans de bonnes conditions et d'en tirer le meilleur parti sur le plan pédagogique.

Je souhaite aussi remercier l'ensemble de l'équipe du service Archivage de la SNRT pour son accueil, sa disponibilité et le partage de sa connaissance du terrain, qui a grandement contribué à la compréhension des besoins réels ayant guidé la conception de la plateforme présentée dans ce rapport.

Enfin, je remercie les membres du jury pour l'intérêt qu'ils ont porté à ce travail et pour le temps consacré à son évaluation.



Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage effectué au sein du département Radio, service Archivage, de la Société Nationale de Radiodiffusion et de Télévision (SNRT). Ce stage a porté sur la conception et le développement d'une plateforme de recherche et d'interrogation intelligente des archives audiovisuelles et documentaires de la SNRT, composées de contenus textuels et audio en plusieurs langues (arabe, français).

La solution développée repose sur une architecture en microservices, orchestrée avec Docker, combinant un pipeline d'ingestion automatique (extraction de texte, transcription audio avec Whisper, classification parole/musique avec YAMNet, enrichissement par modèle de langage via Ollama), une double indexation – plein texte avec Apache Solr et sémantique avec ChromaDB – ainsi qu'un assistant conversationnel fondé sur une architecture de génération augmentée par récupération (RAG). L'ensemble des traitements, y compris ceux relevant de l'intelligence artificielle, s'exécute intégralement en local, sans recours à des API externes, répondant ainsi à l'exigence de confidentialité des données propre à un établissement public audiovisuel.

Une interface web complète, développée avec React et une API FastAPI, permet la recherche, la consultation, l'édition du contenu extrait, et le suivi de l'activité des utilisateurs. Le travail a également porté sur la mise en place de pratiques DevOps – conteneurisation avec contrôle de santé des services, intégration continue et supervision applicative – afin d'assurer la fiabilité et la maintenabilité de l'ensemble du système.

Mots clés : Traitement automatique de la parole, Intelligence artificielle, Retrieval-Augmented Generation, Microservices, Recherche sémantique, Docker.

Liste des Abréviations

Abréviation	**Désignation**
API	Application Programming Interface
BM25	Best Matching 25 (algorithme de pertinence textuelle)
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
SNRT	Société Nationale de Radiodiffusion et de Télévision
UML	Unified Modeling Language
YAML	YAML Ain't Markup Language

Introduction Générale

La Société Nationale de Radiodiffusion et de Télévision (SNRT) produit et diffuse quotidiennement un volume important de contenus audiovisuels et documentaires. Au fil des années, cette production a donné lieu à des archives considérables, mêlant documents textuels et enregistrements audio, dans plusieurs langues, dont l'exploitation repose aujourd'hui encore largement sur des méthodes manuelles. Cette situation limite fortement la capacité des équipes de la SNRT à retrouver rapidement une information précise au sein de ce patrimoine, qu'il s'agisse d'un sujet déjà traité, d'une déclaration ponctuelle ou d'un enregistrement particulier.

C'est dans ce contexte que s'inscrit le stage effectué au sein du département Radio, service Archivage, de la SNRT, dont l'objectif a été de concevoir et de développer une plateforme de recherche et d'interrogation intelligente de ces archives, capable de traiter conjointement des contenus textuels et audio, dans un cadre respectant une exigence essentielle posée par l'entreprise : l'exécution intégralement locale de l'ensemble des traitements, y compris ceux relevant de l'intelligence artificielle, sans recours à des services externes susceptibles de compromettre la confidentialité des données.

La solution développée repose sur un pipeline automatique de traitement des fichiers – transcription audio, détection de langue, classification parole/musique, enrichissement par modèle de langage – couplé à une double indexation, plein texte et sémantique, ainsi qu'un assistant conversationnel permettant d'interroger les archives en langage naturel. L'ensemble est exposé à travers une interface web complète, développée selon une démarche itérative, brique par brique, et accompagnée de pratiques DevOps visant à en garantir la fiabilité.

Le présent rapport est structuré en cinq chapitres :

— **Chapitre 1 – Contexte du stage :** présentation de la SNRT, problématique, étude des solutions existantes, objectifs et planification du stage.

— **Chapitre 2 – Analyse des besoins :** identification des acteurs, besoins fonctionnels et non fonctionnels, et diagramme de cas d'utilisation.

— **Chapitre 3 – Conception et architecture :** diagramme de séquence, modèle de données, architecture physique et logicielle, choix

technologiques.

— **Chapitre 4 – Pipeline IA :** transcription et détection de langue, classification audio, enrichissement par modèle de langage, architecture de l'assistant conversationnel (RAG), et difficultés rencontrées.

— **Chapitre 5 – Réalisation :** outils utilisés, présentation des interfaces développées, et pratiques DevOps mises en place.

Table des matières

Remerciements **2**

Résumé **3**

Remerciements **3**

Liste des Abréviations **4**

Résumé **4**

Introduction Générale **5**

Liste des Abréviations **5**

Introduction générale **7**

1 Contexte du stage **13**

1.1 Introduction 14

1.2 Présentation de l'organisme d'accueil 14

1.2.1 Présentation générale 14

1.2.2 Historique 14

1.2.3 Missions 15

1.2.4 Organisation et offre de services 15

1.2.5 Cadre du stage 15

1.3 Cadrage du projet 16

1.3.1 Problématique 16

1.3.2 Étude des solutions existantes 17

1.3.3 Objectifs du projet 19

1.4 Méthodologie de travail 19

1.5 Planification du stage 21

1.5.1 Cycle de vie du projet 21

1.5.2 Planification : diagramme de Gantt 21

1.5.3 Planification : diagramme de Gantt 21

1.6 Conclusion 22

2 Analyse des besoins 23
2.1 Introduction 24
2.2 Identification des acteurs 24
2.3 Spécification des besoins fonctionnels 25
2.4 Spécification des besoins non fonctionnels 27
2.5 Diagrammes de cas d'utilisation 27
2.5.1 Diagramme de cas d'utilisation global 27
2.5.2 Description textuelle des cas d'utilisation 28
2.6 Conclusion 30

3 Conception et architecture 31
3.1 Introduction 32
3.2 Architecture logicielle 32
3.2.1 Choix d'une architecture en microservices 32
3.2.2 Vue d'ensemble de l'architecture 32
3.3 Flux de données : pipeline d'ingestion 34
3.3.1 Vue générale du pipeline 34
3.3.2 Description du flux 34
3.4 Modèle de données 35
3.4.1 Choix de MongoDB 35
3.4.2 Collections principales 36
3.4.3 Indexation complémentaire 37
3.5 Choix technologiques 37
3.6 Conclusion 39

4 Pipeline IA 40
4.1 Introduction 41
4.2 Transcription et détection de langue 41
4.2.1 Choix du modèle Whisper 41
4.2.2 Détection fiable de la langue 42
4.2.3 Motivation 43
4.2.4 Modèle retenu : YAMNet 43
4.3 Enrichissement par modèle de langage 45
4.4 Assistant conversationnel : architecture RAG 46
4.4.1 Principe général 46
4.4.2 Découpage des documents en fragments 47
4.4.3 Indexation vectorielle 48

4.4.4 Déduplication des sources 48
4.4.5 Génération de la réponse et prompt 49
4.4.6 Contraintes de performance 50
4.5 Difficultés rencontrées et solutions 50
4.6 Conclusion 51

5 Réalisation **52**
5.1 Introduction 53
5.2 Outils utilisés 53
5.2.1 Environnement de développement 53
5.2.2 Backend et bases de données 53
5.2.3 Frontend 53
5.2.4 Intelligence artificielle 53
5.2.5 Gestion de versions 54
5.3 Présentation des interfaces 54
5.3.1 Authentification 54
5.3.2 Recherche 54
5.3.3 Historique 55
5.3.4 Tableau de bord 56
5.3.5 Éditeur de documents 58
5.4 Pratiques DevOps 59
5.4.1 Contrôles de santé des services 59
5.4.2 Intégration continue 60
5.4.3 Supervision applicative 61
5.5 Conclusion 62

Conclusion Générale **64**

Bibliographie **66**

Conclusion Générale **66**

Annexes **67**

Table des figures

1.1 Logo de la Société Nationale de Radiodiffusion et de Télévision (SNRT) . . . 14
1.2 Incrémentale et itérative . . . 20
1.3 diagramme de Gantt . . . 22

2.1 Diagramme de cas d'utilisation global de la plateforme SNRT Search . . . 28

3.1 Architecture microservices de la plateforme SNRT Search . . . 33
3.2 Diagramme de séquence du pipeline d'ingestion d'un fichier audio 34
3.3 Modèle de données de la plateforme (collections MongoDB) . . 36

4.1 Architecture interne de Whisper (encoder-decoder). Source : OpenAI, `https://openai.com/index/whisper/` . . . 42
4.2 Résultats du service de détection de langue sur deux fichiers audio réels du corpus . . . 43
4.3 Architecture de YAMNet, fondée sur des convolutions séparables en profondeur . . . 44
4.4 Pipeline de traitement audio (flux Whisper + YAMNet) . . . 45
4.5 Architecture RAG (retrieval → augmentation → génération) . 47
4.6 Schéma du chunking à fenêtre glissante) . . . 48
4.7 Réponse de l'assistant conversationnel avec cinq sources distinctes, après correction de la déduplication . . . 49

5.1 Écrans de connexion et d'inscription . . . 54
5.2 Résultats de recherche sur un document français (gauche) et sur un fichier audio arabe (droite) . . . 55
5.3 Interface d'historique, mêlant recherches et conversations . . . 56
5.4 Vue d'ensemble du corpus : répartition par type et par langue, et évolution de l'ingestion dans le temps . . . 57
5.5 Activité personnelle de l'utilisateur : répartition recherche/chat et évolution dans le temps . . . 57
5.6 Éditeur de documents : liste et vue détaillée d'un document texte 58

5.7 Éditeur de documents : liste et vue détaillée d'une transcription audio en arabe 59
5.8 État de santé de l'ensemble des services (`docker compose ps`) 60
5.9 Exécution réussie du pipeline d'intégration continue 61
5.10 Tableau de bord de supervision applicative, douze services opérationnels 62
5.11 Document de travail initial (pistes d'architecture envisagées, non toutes retenues) 68

Liste des tableaux

1.1 Comparatif des approches existantes face aux besoins de la SNRT 18

2.1 Description textuelle détaillée des cas d'utilisation de la plateforme SNRT Search 29

3.1 Synthèse des choix technologiques et de leur justification 38

Chapitre 1

Contexte du stage

1.1 Introduction

Ce premier chapitre présente le cadre général dans lequel s'inscrit le stage. Nous y présentons l'organisme d'accueil, la problématique à l'origine du projet, un aperçu des solutions existantes dans ce domaine, les objectifs fixés pour le stage, ainsi que la méthodologie de travail et la planification adoptées tout au long de la période de stage.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation générale

La **Société Nationale de Radiodiffusion et de Télévision (SNRT)** est l'opérateur public marocain de radiodiffusion et de télévision. Il s'agit d'une société de participation dont le capital est détenu à 100% par l'État marocain. La SNRT est membre actif de l'Union européenne de radio-télévision (UER) et actionnaire de la chaîne d'information internationale Euronews.



FIGURE 1.1 – Logo de la Société Nationale de Radiodiffusion et de Télévision (SNRT)

1.2.2 Historique

Les origines de la SNRT remontent à la création de Radio Maroc, dont les premières émissions débutent en avril 1928, sous la tutelle de l'Office chérifien des postes et télégraphes. L'établissement connaît plusieurs évolutions statutaires au cours du XXe siècle, avant d'être rattaché à l'administration centrale du ministère de l'Information en 1978.

C'est en avril 2005 que la SNRT, telle qu'elle existe aujourd'hui, succède à la Radiodiffusion Télévision Marocaine (RTM) dans l'ensemble de ses moyens et attributions, dans un contexte de libéralisation progressive du secteur audiovisuel marocain. Depuis cette date, l'établissement a engagé une

modernisation continue de ses outils de production et de diffusion, ainsi qu'un développement de son offre de chaînes.

1.2.3 Missions

La SNRT assure le service public de radiodiffusion et de télévision au Maroc, dans le cadre du monopole de l'État en matière de télécommunications audiovisuelles, conformément aux textes législatifs et réglementaires en vigueur. Ses missions couvrent la production et la diffusion de programmes d'information, de culture, de divertissement et de service public, à destination du territoire national ainsi que de la diaspora marocaine.

1.2.4 Organisation et offre de services

La SNRT s'appuie aujourd'hui sur un effectif d'environ 2 356 collaborateurs, répartis entre les fonctions de programmation, de production et de diffusion. Son offre se structure autour de plusieurs pôles :

- **Télévision** : huit chaînes, parmi lesquelles Al Aoula (chaîne généraliste historique), Arryadia (sport), Athaqafia (culture), Assadissa (religieux), Aflam TV (cinéma), Tamazight (culture amazighe), ainsi que des déclinaisons régionales et internationales.
- **Radio** : plusieurs stations nationales (Radio Nationale Marocaine, Chaîne Inter, Radio Amazigh, Radio Mohammed VI du Saint Coran, Casa FM) ainsi qu'un réseau de radios régionales couvrant les principales villes du Royaume.
- **Services numériques** : la SNRT a plus récemment développé une offre numérique, avec notamment SNRT-News (service d'articles de presse en ligne, en arabe et en français) et Forja (plateforme de vidéo à la demande).

Cette diversité de chaînes et de stations, produisant quotidiennement un volume important de contenus, explique en grande partie l'accumulation d'archives audiovisuelles et documentaires évoquée dans la problématique du présent stage.

1.2.5 Cadre du stage

Le stage s'est déroulé au sein du **département Radio, au service Archivage**, direction en charge de la conservation, de l'organisation et de l'exploitation des archives sonores produites par les différentes stations de radio de la SNRT. Ce service constitue un point d'observation direct des difficultés :

volume croissant d'enregistrements, absence de transcription systématique, et recherche manuelle au sein des fonds sonores.

C'est dans ce cadre que s'inscrit le projet mené durant le stage, du 25/06/2026 au 25/08/2026, visant à outiller ce service d'une plateforme de recherche et d'interrogation intelligente de ses archives, couvrant à la fois les contenus audio et les documents textuels associés.

1.3 Cadrage du projet

1.3.1 Problématique

L'exploitation des archives de la SNRT soulève plusieurs difficultés concrètes, identifiées dès le début du stage :

- **Volume et hétérogénéité des données** : les archives mélangent des documents textuels et des fichiers audio, accumulés depuis la création de l'établissement, dans plusieurs langues (**arabe**, **français**, **anglais**, **amazigh/tifinagh**), sans structuration commune permettant une recherche unifiée. En pratique, les équipes de la SNRT expriment un besoin récurrent et concret : retrouver rapidement une information ponctuelle – par exemple une mission déjà couverte, un sujet déjà traité, ou une déclaration précise – au sein de ce volume, ce qui s'avère aujourd'hui très difficile compte tenu de la quantité de données concernée et de leur diversité linguistique. Cette diversité rend en particulier toute solution de recherche mono-langue insuffisante.
- **Absence de transcription du contenu audio** : les fichiers audio ne sont pas transcrits en texte, ce qui les rend invisibles à toute recherche par mot-clé et empêche leur indexation.
- **Recherche peu efficace** : sans indexation adaptée, retrouver un document ou un passage audio précis nécessite une recherche manuelle longue et fastidieuse.
- **Absence d'outil d'interrogation en langage naturel** : aucun outil ne permet de poser une question directement sur le contenu des archives et d'obtenir une réponse synthétique accompagnée de ses sources.
- **Exigence de confidentialité des données** : en tant qu'établissement public audiovisuel, la SNRT ne souhaite pas que le contenu de ses archives – potentiellement sensible sur le plan éditorial ou stratégique – transite par des services tiers via Internet. Toute solution reposant sur des API externes de transcription ou de génération de texte était donc exclue d'emblée : l'ensemble des traitements devait pouvoir s'exécuter

intégralement en local, au sein de l'infrastructure de la SNRT, sans dépendance à un service externe.

À cette exigence de confidentialité, propre à l'entreprise, s'est ajoutée une contrainte matérielle spécifique à l'environnement de développement utilisé durant le stage : la machine mise à disposition pour le développement ne disposait pas de GPU dédié. Cette contrainte personnelle au poste de travail du stage a orienté le choix des modèles d'intelligence artificielle vers des architectures légères, capables de s'exécuter avec des temps de traitement raisonnables sur processeur (CPU) seul – ce choix restant cohérent avec l'exigence de traitement local imposée par la SNRT, et non l'inverse.

1.3.2 Étude des solutions existantes

Plusieurs catégories d'outils existent pour répondre partiellement à cette problématique. Le tableau 1.1 en propose une synthèse comparative, selon des critères directement liés aux contraintes du projet : prise en charge de l'audio, capacité d'interrogation en langage naturel, support du multilinguisme, possibilité d'un déploiement local, et coût.

— **Moteurs de recherche documentaire génériques** (Apache Solr, Elasticsearch) : efficaces pour l'indexation et la recherche plein texte, mais ne traitent pas nativement le contenu audio et ne proposent pas de couche conversationnelle.

— **Outils de transcription automatique** (Whisper d'OpenAI, services cloud de speech-to-text) : permettent de convertir l'audio en texte, mais ne constituent qu'une brique isolée d'un pipeline complet et ne gèrent pas la classification du contenu, l'indexation, ni l'exposition d'une interface utilisateur.

— **Solutions de type RAG (Retrieval-Augmented Generation)** : de plus en plus répandues pour interroger des bases documentaires en langage naturel, mais généralement conçues pour du contenu déjà textuel et rarement adaptées à un pipeline d'ingestion audio de bout en bout.

Solution	Audio	Langage naturel	Multilingue	Local	Coût
Moteurs de recherche documentaire (Solr, Elasticsearch)	Non	Non	Partiel	Oui	Gratuit
Outils de transcription seuls (Whisper, STT cloud)	Oui	Non	Oui	Selon solution	Variable
Solutions RAG génériques (sur contenu déjà textuel)	Non	Oui	Partiel	Selon solution	Variable
API LLM propriétaires (OpenAI, Google, etc.)	Oui [1]	Oui	Oui	**Non**	Payant
Solution proposée (ce stage)	**Oui**	**Oui**	**Oui**	**Oui**	**Gratuit**

TABLE 1.1 – Comparatif des approches existantes face aux besoins de la SNRT

Via des API de transcription et de génération séparées, proposées par le même fournisseur.

Les **API propriétaires de grands modèles de langage** (telles que celles proposées par OpenAI, Google ou d'autres fournisseurs cloud) constituent aujourd'hui une option technique particulièrement performante et rapide à mettre en œuvre : elles permettent, avec un minimum de développement, de bénéficier de capacités de transcription, de résumé et de réponse en langage naturel de très bonne qualité, sur un grand nombre de langues.

Cette option a néanmoins été écartée dès le cadrage du projet. Son usage impliquerait en effet de transmettre à un prestataire tiers l'intégralité du contenu des archives – y compris des contenus audiovisuels sensibles ou non encore diffusés – via Internet, pour chaque opération de transcription ou de génération de réponse. Or, la SNRT, en tant qu'établissement public audiovisuel, ne peut accepter qu'une donnée interne, potentiellement sensible sur le plan éditorial ou stratégique, quitte le périmètre de ses propres infrastructures. Cette contrainte de confidentialité constitue ainsi un critère de conception à part entière du projet, au même titre que la contrainte matérielle (absence de GPU) déjà évoquée : l'ensemble des traitements – transcription, classification, génération de résumés, recherche conversationnelle – devait pouvoir s'exécuter **intégralement en local**, sans dépendance à un service externe ni transfert de données hors de l'infrastructure de la SNRT.

C'est ce double impératif – traitement local et absence de coût récurrent lié à des appels API – qui a orienté l'ensemble des choix technologiques du projet vers des outils open source exécutables sur CPU .

1.3.3 Objectifs du projet

Le stage vise les objectifs suivants :

1. Concevoir un pipeline d'ingestion automatique capable de détecter, traiter et indexer aussi bien des documents textuels que des fichiers audio.
2. Intégrer un module de transcription automatique de la parole (speech-to-text), avec détection de la langue.
3. Distinguer automatiquement, au sein des fichiers audio, les passages relevant de la parole de ceux relevant de la musique, afin d'orienter chaque type de contenu vers le traitement approprié.
4. Enrichir automatiquement chaque document (résumé, mots-clés, thématiques) à l'aide d'un modèle de langage.
5. Mettre en place un moteur de recherche plein texte sur l'ensemble du corpus indexé.
6. Développer un assistant conversationnel (RAG) permettant d'interroger les archives en langage naturel, avec citation des sources utilisées pour chaque réponse.
7. Concevoir une interface web complète permettant la recherche, la consultation, l'édition et le suivi de l'activité des utilisateurs.
8. Assurer la fiabilité et la maintenabilité de l'ensemble du système à travers des pratiques DevOps (conteneurisation, supervision, intégration continue).

1.4 Méthodologie de travail

Le développement du projet a suivi une démarche **incrémentale et itérative**, brique par brique, plutôt qu'une planification figée en amont. Chaque fonctionnalité a été développée, testée et validée individuellement avant l'ajout de la suivante, selon le cycle suivant :

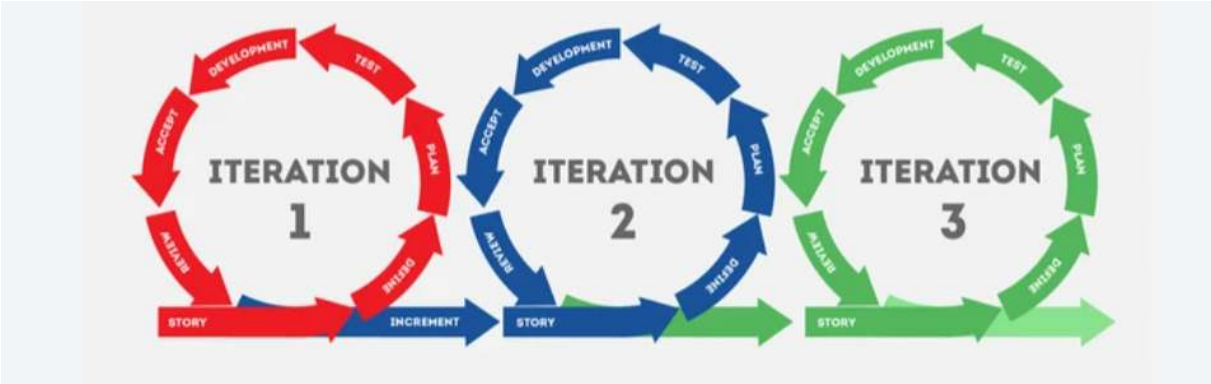


FIGURE 1.2 – Incrémentale et itérative

1. Spécification d'une fonctionnalité précise et délimitée (par exemple : détection de la langue d'un fichier audio).
2. Implémentation de la brique correspondante, sous forme d'un microservice indépendant lorsque cela était pertinent.
3. Test unitaire de la brique via des requêtes directes (`curl`), avant toute intégration avec le reste du système.
4. Intégration de la brique dans le pipeline ou l'interface existante.
5. Test de bout en bout, incluant la vérification des journaux d'exécution (*logs*) de chaque conteneur concerné.
6. Correction des anomalies identifiées avant de passer à la fonctionnalité suivante.

Cette approche a permis de limiter la complexité à chaque étape, de détecter rapidement les régressions, et de garder à tout moment un système fonctionnel sur lequel s'appuyer pour la suite du développement. Elle a également facilité le diagnostic d'anomalies, plusieurs incidents ayant été rencontrés et résolus au cours du stage (voir section 4.5 du chapitre 4), tels que des conflits de versions de dépendances, des routes dupliquées dans le code, ou des healthchecks Docker mal configurés.

Sur le plan des outils, le travail a été organisé autour de :

— un dépôt Git hébergé sur GitHub, avec des messages de commit décrivant précisément chaque changement apporté ;
— une architecture en microservices, chacun disposant de son propre dépôt de code, de son `Dockerfile` et de ses dépendances (`requirements.txt`) ;
— un environnement Docker Compose permettant d'orchestrer l'ensemble des services et de reproduire l'environnement complet sur n'importe quelle machine.

1.5 Planification du stage

1.5.1 Cycle de vie du projet

Le projet s'est déroulé selon les grandes phases suivantes :

1. **Prise en main et cadrage** : découverte de l'environnement existant (services déjà en place : API de recherche, service de transcription, service LLM, indexation Solr), identification des axes d'amélioration.
2. **Consolidation de l'existant** : vérification et correction du pipeline de recherche et de chat déjà en place.
3. **Enrichissement du pipeline audio** : ajout de la détection fiable de la langue, de la conservation des horodatages de transcription, puis de la classification parole/musique.
4. **Développement des fonctionnalités utilisateur** : authentification, historique, tableau de bord, éditeur de documents.
5. **Mise en place des pratiques DevOps** : healthchecks, intégration continue, supervision applicative.
6. **Rédaction du rapport de stage.**

1.5.2 Planification : diagramme de Gantt

Le diagramme ci-dessous synthétise la répartition temporelle des différentes phases du stage.

1.5.3 Planification : diagramme de Gantt

Le diagramme ci-dessous synthétise la répartition temporelle des différentes phases du stage.

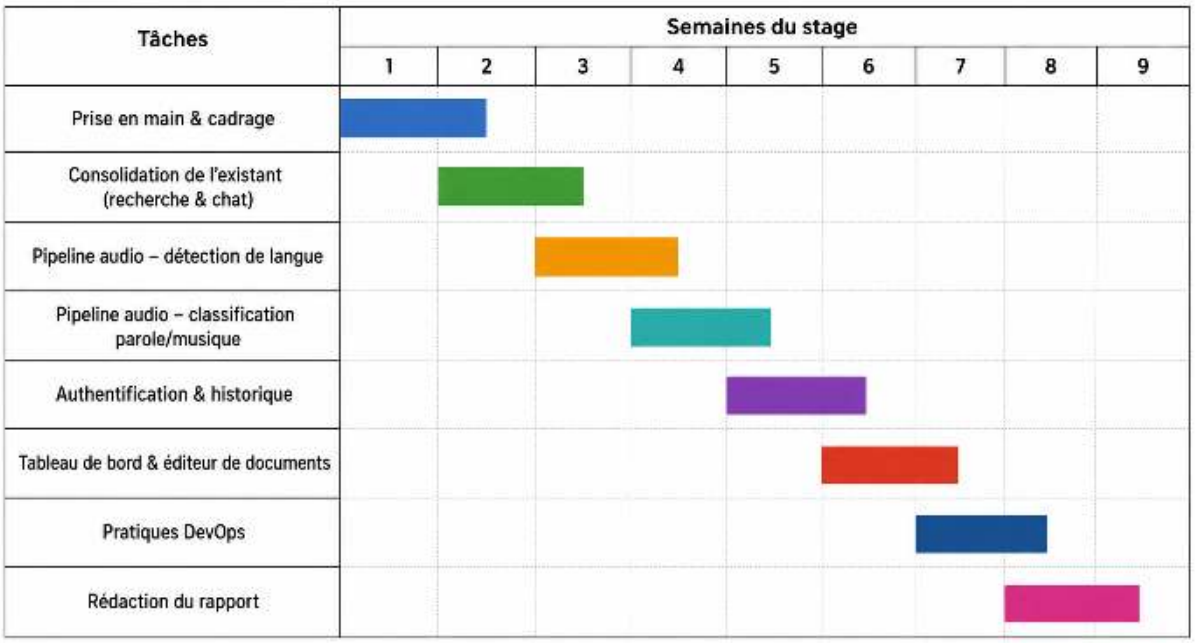


FIGURE 1.3 – diagramme de Gantt

1.6 Conclusion

Ce chapitre a permis de poser le cadre général du stage : la présentation de la SNRT, la problématique liée à l'exploitation de ses archives audiovisuelles et documentaires, les limites des solutions existantes, ainsi que les objectifs et la méthodologie de travail adoptée. Le chapitre suivant détaille l'analyse des besoins fonctionnels et non fonctionnels ayant guidé la conception de la solution.

Chapitre 2

Analyse des besoins

2.1 Introduction

Avant d'entamer la conception technique de la plateforme, il est nécessaire de formaliser précisément ce que le système doit accomplir et pour qui. C'est l'objet de ce chapitre, qui constitue la charnière entre le cadrage général présenté au chapitre précédent et les choix d'architecture détaillés au chapitre suivant.

Nous procédons ici en quatre temps. Nous identifions d'abord les acteurs appelés à interagir avec le système, qu'ils soient humains (l'utilisateur final) ou automatisés (le processus d'ingestion). Nous spécifions ensuite les besoins fonctionnels, c'est-à-dire l'ensemble des actions que le système doit permettre de réaliser, regroupées par domaine fonctionnel (authentification, recherche, dialogue conversationnel, historique, tableau de bord, édition de contenu et ingestion automatique). Nous formulons également les besoins non fonctionnels, qui portent non pas sur *ce que* le système fait, mais sur *comment* il doit le faire : contraintes de performance, de sécurité, de disponibilité et de maintenabilité. Enfin, nous synthétisons l'ensemble de cette analyse à travers un diagramme de cas d'utilisation UML, accompagné d'une description textuelle détaillée de chaque cas identifié, incluant les relations d'inclusion (`«include»`) entre certains cas d'utilisation et les mécanismes techniques sous-jacents qu'ils mobilisent.

Cette analyse des besoins a été menée de façon progressive au cours du stage, chaque nouvelle fonctionnalité ayant fait l'objet d'une réflexion sur son utilité réelle avant son implémentation — conformément à la démarche itérative décrite au chapitre précédent. Le résultat présenté ci-après reflète donc l'ensemble du périmètre fonctionnel finalement retenu et implémenté.

2.2 Identification des acteurs

L'analyse du système fait apparaître deux acteurs principaux :

— **Utilisateur authentifié** : toute personne appartenant au département des archives radio de la SNRT et disposant d'un compte sur la plateforme. Il interagit directement avec l'interface web pour rechercher des fichiers, dialoguer avec l'assistant conversationnel, déposer de nouveaux fichiers, consulter son historique, visualiser des statistiques et corriger le contenu extrait des fichiers indexés.

— **Système d'ingestion (acteur automatisé)** : ce n'est pas un utilisateur humain, mais un processus autonome (le service `watcher`) qui

surveille en permanence l'arrivée de nouveaux fichiers et déclenche automatiquement leur traitement (classification, transcription, analyse, indexation), sans intervention humaine.

À ce stade du projet, un seul niveau d'habilitation a été mis en place. Tous les utilisateurs authentifiés du département des archives radio de la SNRT disposent des mêmes droits d'accès et peuvent utiliser l'ensemble des fonctionnalités de la plateforme.

2.3 Spécification des besoins fonctionnels

Les besoins fonctionnels identifiés sont regroupés par domaine :

Authentification

- L'utilisateur peut créer un compte à l'aide d'une adresse e-mail et d'un mot de passe.
- L'utilisateur peut se connecter et rester authentifié entre deux sessions (persistance du jeton de connexion).
- L'utilisateur peut se déconnecter à tout moment.

Dépôt et recherche de fichiers

- L'utilisateur peut déposer un nouveau fichier (document ou audio), qui est ensuite pris en charge par le système d'ingestion.
- L'utilisateur peut rechercher un terme dans l'ensemble du corpus (documents textuels et transcriptions audio) au moyen d'une recherche plein texte.
- L'utilisateur peut filtrer les résultats par type de contenu (document ou audio) et par langue.
- Les résultats affichent un extrait pertinent, un score de correspondance, ainsi que les métadonnées du fichier (résumé, thématiques, mots-clés).

Assistant conversationnel (Chat)

- L'utilisateur peut poser une question en langage naturel portant sur le contenu des archives.
- Le système effectue une recherche sémantique parmi les fichiers vectorisés afin d'identifier les passages les plus pertinents.
- Le système génère une réponse synthétique fondée sur ces passages, à l'aide d'un modèle de langage.

— Chaque réponse est accompagnée de la liste des fichiers sources ayant permis de la générer.
— Le système indique explicitement lorsqu'il ne dispose pas d'une information suffisante pour répondre.

Historique d'activité

— Chaque recherche et chaque échange conversationnel sont automatiquement enregistrés.
— L'utilisateur peut consulter, filtrer et supprimer les entrées de son historique.
— L'utilisateur peut relancer une recherche précédente en un clic depuis son historique.
— L'historique et les conversations sont conservés de façon persistante, indépendamment des reconnexions ou changements de page.

Tableau de bord (Dashboard)

— L'utilisateur peut visualiser des statistiques globales sur le corpus (répartition par type de contenu, par langue, évolution de l'ingestion dans le temps).
— L'utilisateur peut visualiser ses statistiques personnelles d'activité (nombre de recherches, de questions posées, répartition dans le temps).

Édition de l'extraction d'un fichier

— L'utilisateur peut consulter la liste des fichiers (documents et audio) indexés.
— L'utilisateur peut consulter le détail d'un fichier (texte extrait, résumé, thématiques, mots-clés).
— L'utilisateur peut corriger manuellement le texte extrait, le résumé, les thématiques et les mots-clés d'un fichier, puis enregistrer ses modifications.
— Lorsqu'un fichier audio est identifié comme un contenu musical, l'interface l'indique explicitement et ne propose pas d'édition de transcription, celle-ci n'existant pas pour ce type de contenu.

Ingestion automatique (système)

— Le système détecte automatiquement l'ajout de tout nouveau fichier dans le répertoire surveillé.

— Pour un document textuel, le système en extrait le texte, l'analyse, puis l'indexe.
— Pour un fichier audio, le système détermine d'abord s'il s'agit de parole ou de musique.
— Si le contenu est de la parole, le système le transcrit, détecte sa langue, l'analyse puis l'indexe.
— Si le contenu est musical, le système l'enregistre sans le transcrire ni l'indexer en texte plein.

2.4 Spécification des besoins non fonctionnels

— **Compatibilité matérielle** : l'ensemble des traitements, y compris les modèles d'intelligence artificielle, doivent pouvoir s'exécuter sur une machine ne disposant pas de GPU dédié.
— **Sécurité** : les mots de passe des utilisateurs sont stockés sous forme hachée (*bcrypt*) et l'accès aux ressources personnelles (historique, favoris) est protégé par authentification à jeton (JWT).
— **Modularité** : chaque fonctionnalité majeure est isolée dans un microservice indépendant, déployable et testable séparément.
— **Disponibilité et fiabilité** : chaque service expose un point de contrôle de santé (*healthcheck*), permettant de superviser l'état du système en continu.
— **Maintenabilité** : le code est organisé par service, versionné avec Git, et soumis à une vérification automatique (intégration continue) avant toute fusion de modification.
— **Performance perçue** : les temps de réponse des fonctionnalités interactives (recherche, chat) doivent rester compatibles avec un usage confortable, malgré l'absence d'accélération matérielle.
— **Support multilingue** : le système doit traiter de façon fiable des contenus en langue arabe et en langue française.

2.5 Diagrammes de cas d'utilisation

2.5.1 Diagramme de cas d'utilisation global

La figure 2.1 présente une vue d'ensemble des interactions entre les acteurs identifiés et les principales fonctionnalités du système. Deux relations d'inclusion (`«include»`) y sont représentées : le cas *Rechercher un fichier* inclut systématiquement une *recherche plein texte*, tandis que le cas *Dialoguer avec*

le chat inclut une *recherche sémantique*, elle-même suivie d'une génération de réponse par un modèle de langage.

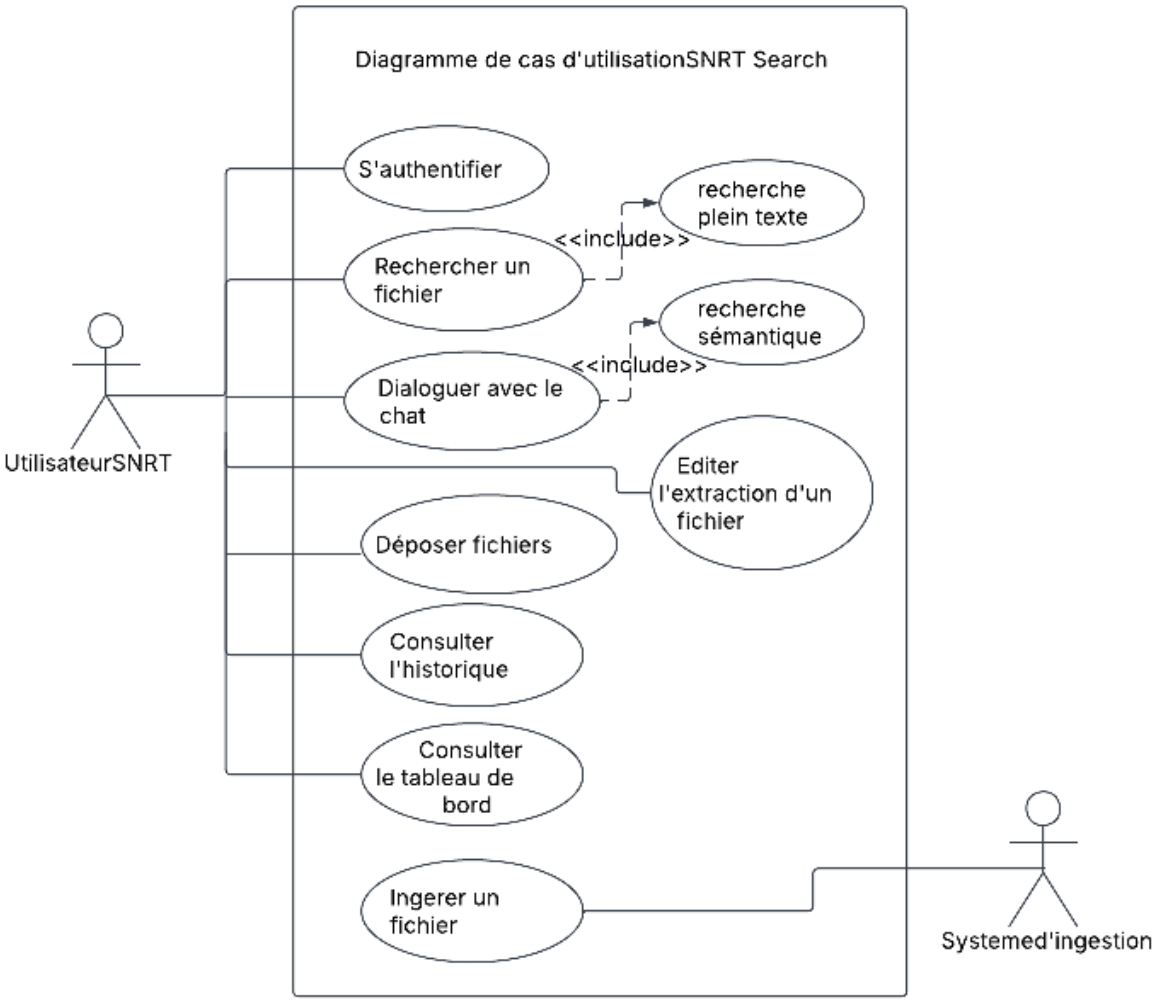


FIGURE 2.1 – Diagramme de cas d'utilisation global de la plateforme SNRT Search

2.5.2 Description textuelle des cas d'utilisation

Le tableau 2.1 détaille, pour chaque cas d'utilisation identifié sur le diagramme, l'acteur concerné, une description de son déroulement, ainsi que le ou les cas d'utilisation inclus lorsque cela s'applique.

Cas d'utilisation	**Acteur**	**Description**	**Cas inclus («include»)**
S'authentifier	Utilisateur	L'utilisateur crée un compte à l'aide d'une adresse e-mail et d'un mot de passe, ou se connecte à un compte existant, afin d'accéder à l'ensemble des fonctionnalités de la plateforme.	—
Rechercher un fichier	Utilisateur	L'utilisateur saisit un terme de recherche, éventuellement filtré par type de contenu (document ou audio) et par langue, et consulte les résultats retournés, classés par pertinence, avec extraits et métadonnées associées.	Recherche plein texte (indexation Solr)
Dialoguer avec le chat	Utilisateur	L'utilisateur pose une question en langage naturel et reçoit une réponse synthétique, générée à partir des fichiers les plus pertinents du corpus, accompagnée de leurs références en tant que sources.	Recherche sémantique (embeddings ChromaDB) ; génération de la réponse (modèle de langage via Ollama)
Déposer fichiers	Utilisateur	L'utilisateur dépose un nouveau fichier (document ou audio) destiné à être traité et intégré au corpus indexé par la plateforme.	—
Éditer l'extraction d'un fichier	Utilisateur	L'utilisateur consulte le contenu extrait d'un fichier (texte, résumé, thématiques, mots-clés) et corrige manuellement ces éléments en cas d'erreur d'extraction ou d'analyse automatique.	—
Consulter l'historique	Utilisateur	L'utilisateur consulte, filtre, rejoue ou supprime les recherches et conversations qu'il a précédemment effectuées sur la plateforme.	—
Consulter le tableau de bord	Utilisateur	L'utilisateur visualise des statistiques globales relatives au corpus indexé (répartition par type et par langue, évolution temporelle), ainsi que des statistiques relatives à sa propre activité.	—
Ingérer un fichier	Système d'ingestion	Le système détecte automatiquement l'arrivée d'un nouveau fichier, détermine sa nature (document, parole ou musique), le traite en conséquence (extraction, transcription, classification, analyse), puis l'indexe le cas échéant dans les moteurs de recherche plein texte et sémantique.	—

TABLE 2.1 – Description textuelle détaillée des cas d'utilisation de la plateforme SNRT Search

2.6 Conclusion

Ce chapitre a permis de traduire la problématique et les objectifs présentés au chapitre précédent en un ensemble structuré de besoins. L'identification des deux acteurs du système — l'utilisateur humain et le processus d'ingestion automatisé — a permis de circonscrire précisément le périmètre fonctionnel de la plateforme, regroupé en sept domaines : authentification, dépôt et recherche de fichiers, dialogue conversationnel, historique, tableau de bord, édition de contenu, et ingestion automatique.

Au-delà des fonctionnalités elles-mêmes, la formulation des besoins non fonctionnels a mis en évidence des contraintes déterminantes pour la suite du projet, en particulier l'impossibilité de recourir à une accélération matérielle (GPU), qui a orienté plusieurs choix technologiques présentés dans les chapitres suivants (choix de modèles légers, architecture en microservices facilitant la répartition de la charge, mise en place de mécanismes de supervision).

Enfin, la formalisation de ces besoins sous la forme d'un diagramme de cas d'utilisation, complété par une description textuelle détaillée des relations d'inclusion entre cas d'utilisation, constitue le point de départ direct de la conception présentée au chapitre 3 : c'est en effet à partir de ce diagramme que sont dérivés l'architecture logicielle en microservices, le modèle de données, ainsi que les flux d'échange entre les différents composants du système.

Chapitre 3

Conception et architecture

3.1 Introduction

Ce chapitre traduit en choix de conception les besoins fonctionnels et non fonctionnels identifiés au chapitre précédent. Il présente successivement l'architecture logicielle retenue, le flux de données qui traverse le système depuis l'arrivée d'un fichier jusqu'à sa disponibilité en recherche, le modèle de données sous-jacent, et enfin les principaux choix technologiques opérés, justifiés au regard des contraintes du projet – en particulier l'absence d'accélération matérielle et l'impératif de traitement local des données.

3.2 Architecture logicielle

3.2.1 Choix d'une architecture en microservices

La plateforme est conçue selon une **architecture en microservices** : chaque fonctionnalité majeure est isolée dans un service indépendant, disposant de son propre code source, de son propre `Dockerfile` et de ses propres dépendances, et communiquant avec les autres services exclusivement via des appels HTTP au sein d'un réseau Docker interne.

Ce choix répond directement à plusieurs contraintes identifiées au chapitre 2 :

— **Modularité** : chaque service peut être développé, testé et redéployé indépendamment des autres, ce qui a permis la démarche itérative décrite au chapitre 1 (une brique = un service = un cycle de test complet).

— **Répartition de la charge** : les services aux besoins en ressources hétérogènes (par exemple le service de transcription Whisper, gourmand en calcul, face à l'API de recherche, légère) peuvent être supervisés, et le cas échéant redéployés ou dimensionnés, indépendamment les uns des autres.

— **Résilience** : la défaillance d'un service isolé (par exemple une indisponibilité temporaire du service de génération de réponses) n'interrompt pas, par construction, le fonctionnement des autres services indépendants, tels que la recherche plein texte.

3.2.2 Vue d'ensemble de l'architecture

La figure 4.6 présente l'ensemble des microservices composant la plateforme, organisés selon les grandes étapes du traitement introduites au chapitre 1 : *extraction*, *indexation* et *restitution* (recherche et dialogue conversationnel).

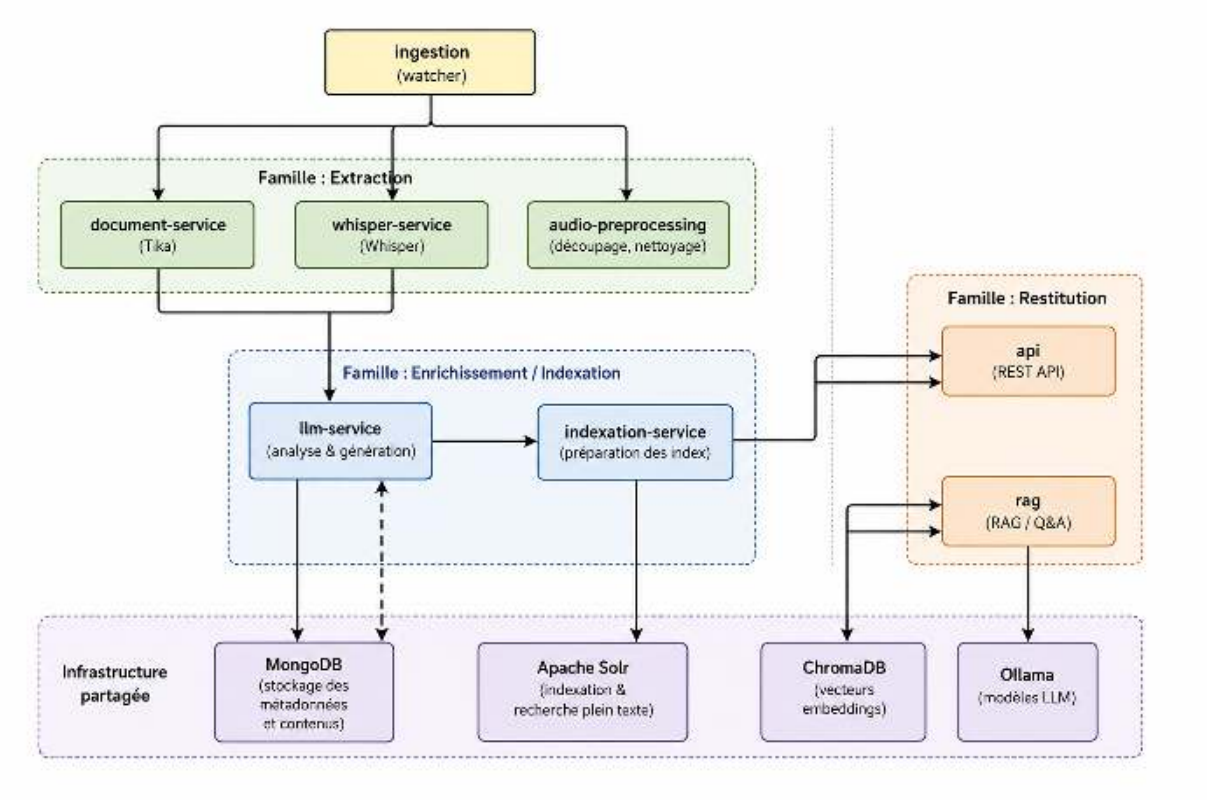


FIGURE 3.1 – Architecture microservices de la plateforme SNRT Search

L'architecture distingue quatre familles de services :

1. **Services d'extraction** : `document-service` (extraction de texte via Apache Tika), `whisper-service` (transcription et détection de langue), et `audio-preprocessing` (classification parole/musique via YAMNet).
2. **Services d'enrichissement et d'indexation** : `llm-service` (génération de résumés, thématiques et mots-clés via un modèle de langage local) et `indexation-service` (indexation dans Apache Solr).
3. **Services de restitution** : `api` (recherche plein texte, authentification, historique) et `rag` (recherche sémantique et génération de réponses conversationnelles).
4. **Service d'orchestration** : `ingestion` (le *watcher*), qui détecte l'arrivée de nouveaux fichiers et orchestre leur passage à travers les services précédents.

Ces services s'appuient sur un ensemble de briques d'infrastructure partagées : MongoDB (stockage documentaire), Apache Solr (indexation plein texte), ChromaDB (indexation vectorielle) et Ollama (exécution locale des modèles de langage et d'embedding).

3.3 Flux de données : pipeline d'ingestion

3.3.1 Vue générale du pipeline

L'ensemble du traitement d'un fichier, depuis son dépôt jusqu'à son indexation en recherche plein texte, est orchestré par le service `ingestion`. La figure 3.2 détaille ce flux sous la forme d'un diagramme de séquence, pour le cas d'un fichier audio – le cas le plus complet, incluant l'étape de classification parole/musique absente du traitement d'un document textuel.

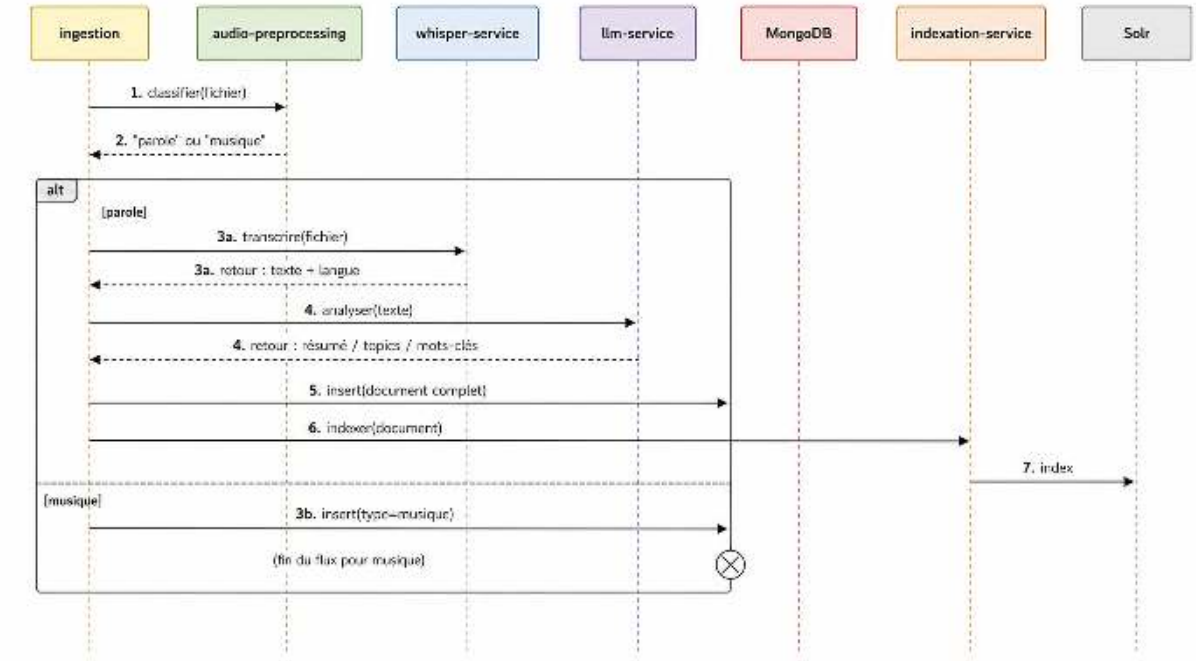


FIGURE 3.2 – Diagramme de séquence du pipeline d'ingestion d'un fichier audio

3.3.2 Description du flux

Le traitement se déroule selon les étapes suivantes :

1. Le service `ingestion` détecte, par surveillance continue du répertoire partagé, l'arrivée d'un nouveau fichier, et détermine sa nature (document ou audio) à partir de son extension.
2. **Cas d'un document textuel** : le fichier est transmis au `document-service`, qui en extrait le texte brut au moyen d'Apache Tika.
3. **Cas d'un fichier audio** : le fichier est d'abord transmis à `audio-preprocessing`, qui détermine s'il s'agit de parole ou de musique.
   — Si le contenu est de la parole, il est transmis au `whisper-service`, qui produit une transcription textuelle ainsi qu'une détection fiable de la langue, fondée sur l'analyse du signal audio lui-même.

   - Si le contenu est identifié comme musical, il est enregistré avec cette classification, sans passer par l'étape de transcription ni par les étapes suivantes d'indexation en texte plein.

4. Le texte obtenu (document ou transcription) est transmis au `llm-service`, qui génère un résumé, une liste de thématiques et une liste de mots-clés à l'aide d'un modèle de langage exécuté localement via Ollama.
5. L'ensemble de ces informations est persisté dans MongoDB, au sein de la collection appropriée (`documents` ou `audio_transcriptions`).
6. Le contenu est transmis à l'`indexation-service`, qui l'indexe dans Apache Solr pour la recherche plein texte.

L'indexation dans ChromaDB, utilisée par l'assistant conversationnel pour la recherche sémantique, n'est **pas** intégrée à ce pipeline automatique : elle est assurée par un script de peuplement dédié (`peupler_chroma.py`), exécuté ponctuellement et manuellement, qui lit l'ensemble des documents déjà persistés dans MongoDB, en calcule les représentations vectorielles par fragments (*chunks*) via Ollama, puis les insère dans ChromaDB. Cette séparation reflète l'ordre réel de développement du projet : le pipeline d'ingestion principal (extraction, transcription, indexation Solr) a été mis en place et consolidé en premier, et l'indexation vectorielle a été ajoutée dans un second temps, sous la forme d'une étape de traitement par lot. L'intégration de cette étape comme opération automatique et systématique du pipeline d'ingestion – déclenchée pour chaque nouveau fichier au même titre que l'indexation Solr – constitue une évolution identifiée en conclusion de ce rapport.

Cette orchestration séquentielle, bien que plus simple à mettre en œuvre qu'un système de files de messages asynchrones, s'est révélée suffisante au regard du volume de fichiers traités durant le stage. Une évolution vers une architecture événementielle (par exemple à l'aide d'une file de messages) constitue une autre piste d'amélioration identifiée en conclusion de ce rapport.

3.4 Modèle de données

3.4.1 Choix de MongoDB

Le stockage principal du système repose sur **MongoDB**, une base de données orientée documents. Ce choix se justifie par l'hétérogénéité des données à stocker : un document textuel et une transcription audio partagent une structure commune (texte, résumé, mots-clés, thématiques) tout en comportant des champs spécifiques (par exemple les segments horodatés et le score

de confiance de détection de langue pour l'audio). Le modèle orienté document, plus souple qu'un schéma relationnel strict, permet de faire évoluer la structure des données au fil des itérations du projet – ce qui s'est avéré précieux dans le cadre de la démarche incrémentale suivie durant le stage.

3.4.2 Collections principales

La figure 3.3 présente le modèle de données sous la forme d'un diagramme de classes, faisant apparaître les quatre collections principales et leurs relations.

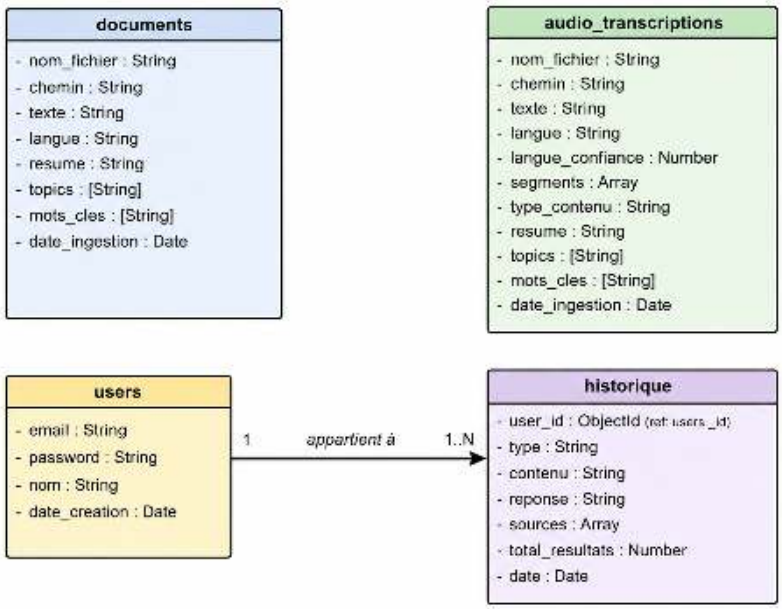


FIGURE 3.3 – Modèle de données de la plateforme (collections MongoDB)

— `documents` : documents textuels ingérés (nom de fichier, chemin, texte extrait, langue, résumé, thématiques, mots-clés, date d'ingestion).
— `audio_transcriptions` : fichiers audio ingérés, structure proche de `documents`, enrichie des champs spécifiques à l'audio (langue détectée et son niveau de confiance, segments horodatés issus de la transcription, classification parole/musique).
— `users` : comptes utilisateurs (adresse e-mail, mot de passe haché, nom, date de création).
— `historique` : entrées d'historique (identifiant de l'utilisateur, type d'action – recherche ou conversation –, contenu, réponse et sources le cas échéant, date). Chaque entrée référence l'utilisateur l'ayant générée par son identifiant, matérialisant ainsi la relation entre les deux collections.

3.4.3 Indexation complémentaire

Au-delà de MongoDB, deux index spécialisés sont maintenus séparément, chacun optimisé pour un type de recherche distinct :

— **Apache Solr** indexe les champs textuels (`texte`, `resume`, `mots_cles`, `topics`) selon l'algorithme BM25, pour la recherche par mots-clés. Cette indexation est déclenchée automatiquement par le pipeline d'ingestion pour chaque nouveau fichier.

— **ChromaDB** indexe les représentations vectorielles des fragments de texte, pour la recherche par similarité sémantique mobilisée par l'assistant conversationnel. Comme précisé en section 3.3.2, cette indexation est alimentée par un script de peuplement exécuté séparément du pipeline automatique.

Ces deux index sont dérivés de MongoDB, qui demeure la source de vérité unique du système : en cas d'incohérence ou de nécessité de réindexation, Solr et ChromaDB peuvent être reconstruits intégralement à partir des données persistées dans MongoDB.

3.5 Choix technologiques

Le tableau 3.1 synthétise les principaux choix technologiques opérés, ainsi que leur justification au regard des contraintes du projet.

Besoin	**Technologie**	**Justification**
Framework API	FastAPI	Framework Python léger et performant, génération automatique d'une documentation interactive (OpenAPI), typage natif via Pydantic, cohérent avec l'écosystème Python déjà utilisé pour les traitements IA.
Stockage principal	MongoDB	Modèle orienté document adapté à l'hétérogénéité des données (textes, audio, métadonnées variables), schéma flexible permettant les évolutions itératives du projet.
Recherche plein texte	Apache Solr	Moteur de recherche mature, algorithme BM25 éprouvé, déploiement local sans dépendance à un service cloud.
Recherche sémantique	ChromaDB	Base vectorielle légère, simple à déployer localement via Docker, suffisante au volume de données traité durant le stage.
Modèle de langage & embeddings	Ollama	Exécution locale d'un modèle de langage open source (`llama3.2:3b`) et d'un modèle d'embedding (`nomic-embed-text`), sans transfert de données vers un service tiers – répondant directement à la contrainte de confidentialité posée au chapitre 1.
Transcription audio	Whisper (OpenAI, open source)	Modèle de reconnaissance vocale multilingue robuste, exécutable sur CPU, disponible en plusieurs tailles permettant un compromis entre qualité et temps de traitement.
Classification audio	YAMNet	Modèle de classification audio léger, pré-entraîné, exécutable sur CPU, suffisant pour la distinction parole/musique visée.
Extraction de documents	Apache Tika	Prise en charge native d'un grand nombre de formats de documents, évitant l'écriture d'un analyseur dédié par format.
Frontend	React	Écosystème mature, architecture à base de composants adaptée à la multiplicité des vues de l'application (recherche, chat, historique, tableau de bord).
Authentification	JWT (JSON Web Token)	Mécanisme d'authentification sans état, mis en œuvre au sein du service `api` (inscription, connexion, protection des routes d'historique et de profil utilisateur), sans nécessiter de session centralisée côté serveur.
Conteneurisation	Docker / Docker Compose	Isolation et reproductibilité de l'environnement de chaque microservice, orchestration simple de l'ensemble du système sur une seule machine de développement.

Table 3.1 – Synthèse des choix technologiques et de leur justification

3.6 Conclusion

Ce chapitre a présenté l'architecture retenue pour répondre aux besoins formalisés au chapitre précédent : une architecture en microservices, organisée autour d'un pipeline d'ingestion séquentiel prenant en charge aussi bien les documents textuels que les fichiers audio, s'appuyant sur un modèle de données orienté document complété par deux index spécialisés – l'un lexical et intégré automatiquement au pipeline, l'autre sémantique et alimenté par un traitement par lot séparé. Les choix technologiques opérés ont été systématiquement guidés par deux contraintes structurantes posées dès le chapitre 1 : l'exécution intégralement locale des traitements, y compris ceux relevant de l'intelligence artificielle, et la compatibilité avec une exécution sur processeur, sans accélération matérielle dédiée. Le chapitre suivant détaille la mise en œuvre concrète de cette architecture, en particulier les composants du pipeline de traitement automatique du contenu audio, cœur technique du projet.

Chapitre 4

Pipeline IA

4.1 Introduction

Ce chapitre détaille le cœur technique du projet, du point de vue de l'ingénierie des données : la chaîne de traitement qui transforme un fichier brut – document ou enregistrement audio – en données structurées, exploitables par les moteurs de recherche et l'assistant conversationnel présentés au chapitre 3.

Quatre briques sont successivement présentées : la transcription et la détection de langue des fichiers audio (Whisper), la classification automatique parole/musique (YAMNet), l'enrichissement du contenu par un modèle de langage (Ollama), et enfin l'architecture de génération augmentée par récupération (RAG) qui constitue l'assistant conversationnel. Chaque brique est présentée avec son principe de fonctionnement, les choix de configuration retenus, et les justifications techniques associées. Le chapitre se conclut par une section dédiée aux principales difficultés rencontrées au cours du stage et aux solutions apportées, dans l'esprit de traçabilité prôné par la démarche itérative décrite au chapitre 1.

4.2 Transcription et détection de langue

4.2.1 Choix du modèle Whisper

La transcription des fichiers audio repose sur **Whisper**, modèle de reconnaissance vocale multilingue publié par OpenAI sous licence open source. Whisper a été retenu pour sa robustesse sur des enregistrements de qualité variable et sa capacité native à traiter plusieurs langues sans modèle dédié par langue – un atout déterminant compte tenu du contexte multilingue des archives de la SNRT (arabe, français, anglais, amazigh).

Compte tenu de la contrainte d'exécution sur CPU, sans accélération matérielle, la variante `tiny` du modèle a été retenue pour l'ensemble des traitements réalisés durant le stage. Ce choix constitue un compromis assumé entre vitesse de traitement et qualité de transcription : les tests effectués ont révélé des approximations notables sur certains passages (mots mal transcrits, répétitions parasites sur des segments de silence ou de musique d'introduction), qui n'affectent cependant pas significativement la pertinence de la recherche, le résumé et les mots-clés générés en aval restant globalement exploitables. L'utilisation d'une variante supérieure (`base` ou `small`) constitue une piste d'amélioration identifiée pour la suite du projet, à condition de disposer de ressources de calcul suffisantes.

La figure 4.1 présente l'architecture interne de Whisper, de type *encoder-decoder* : le signal audio, converti en spectrogramme log-mel, est traité par une pile de blocs *encoder*, dont la représentation est transmise par attention croisée à une pile de blocs *decoder*, produisant la transcription textuelle token par token.

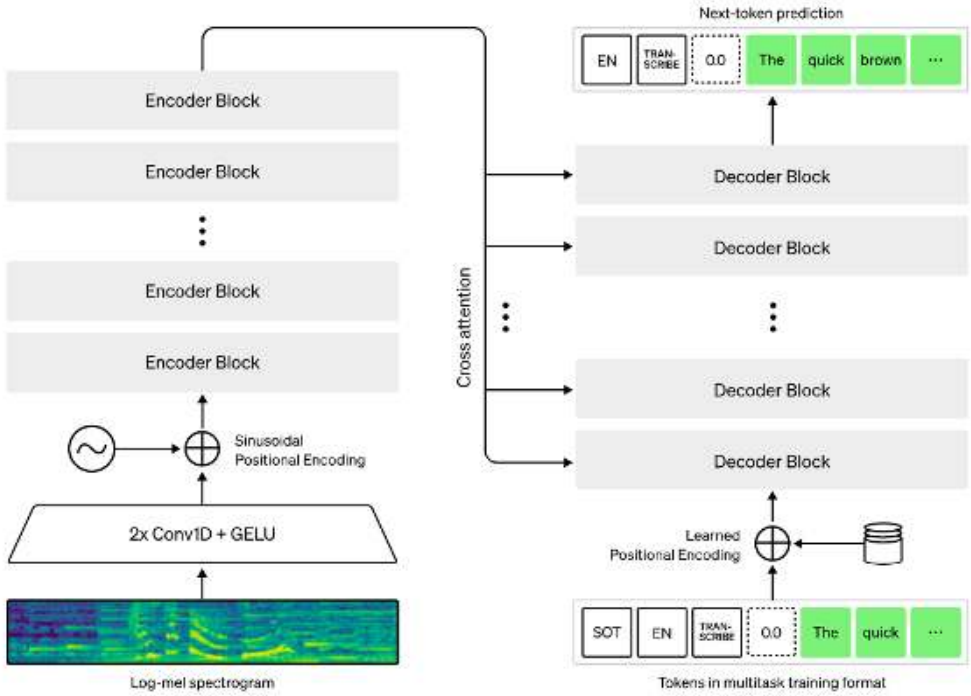


FIGURE 4.1 – Architecture interne de Whisper (encoder-decoder). Source : OpenAI, `https://openai.com/index/whisper/`

4.2.2 Détection fiable de la langue

Une première version du service de transcription déterminait la langue d'un fichier à partir de mots-clés présents dans son nom (*"arabic"*, *"french"*, etc.), une heuristique fragile ne fonctionnant que pour une fraction marginale des fichiers réels, dont les noms ne portent généralement aucune indication de langue.

Cette approche a été remplacée par une détection fondée directement sur le signal audio, à l'aide de la fonction native de détection de langue de Whisper : les premières secondes de l'enregistrement sont converties en spectrogramme, puis soumises au modèle, qui retourne une distribution de probabilités sur l'ensemble des langues qu'il connaît. La langue la plus probable n'est retenue pour orienter la transcription que si son score de confiance dépasse un seuil de 0,5 ; en deçà de ce seuil, la détection est laissée entièrement à la charge de

Whisper pendant la transcription elle-même, évitant ainsi qu'une détection incertaine ne force une langue erronée sur l'ensemble du fichier.

La figure 4.2 présente un test effectué directement sur le service de détection de langue, pour un fichier en arabe et un fichier en français : dans les deux cas, la langue est détectée avec un score de confiance supérieur à 0,97, largement au-dessus du seuil de 0,5 retenu pour forcer la langue de transcription.

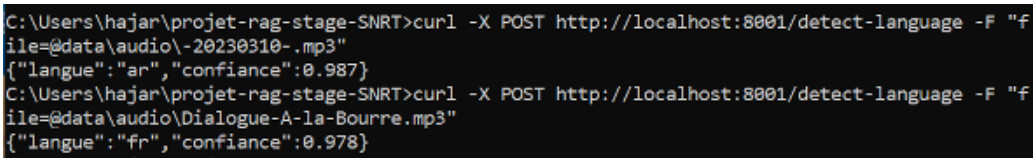


FIGURE 4.2 – Résultats du service de détection de langue sur deux fichiers audio réels du corpus

4.2.3 Motivation

Les archives sonores de la SNRT ne se limitent pas à du contenu parlé : elles comportent également des séquences musicales (jingles, indicatifs, musiques d'illustration). Transmettre un tel contenu à Whisper conduit à des transcriptions dénuées de sens, le modèle tentant d'interpréter un signal musical comme de la parole. Il est donc nécessaire de distinguer, en amont de la transcription, la nature du contenu audio, afin d'orienter chaque fichier vers le traitement approprié : transcription pour la parole, simple archivage sans indexation textuelle pour la musique.

4.2.4 Modèle retenu : YAMNet

Plusieurs approches ont été envisagées pour cette classification, depuis une heuristique fondée sur des caractéristiques acoustiques simples (taux de passage par zéro, centroïde spectral) jusqu'à des modèles pré-entraînés plus complets. Le choix s'est porté directement sur **YAMNet**, réseau de neurones développé par Google, capable de classifier un signal audio parmi 521 catégories sonores, sans qu'une comparaison empirique entre approches n'ait été menée : YAMNet présentait d'emblée l'avantage d'être léger (quelques mégaoctets), exécutable rapidement sur CPU, et de fournir directement des catégories pertinentes pour la distinction recherchée (*Speech*, *Music*, *Silence*, notamment).

La figure 4.3 présente l'architecture de YAMNet, fondée sur des blocs de convolution séparable en profondeur (*depthwise separable convolution*),

une conception qui réduit fortement le nombre de paramètres du modèle par rapport à des convolutions classiques, expliquant sa légèreté et sa rapidité d'exécution sur processeur.

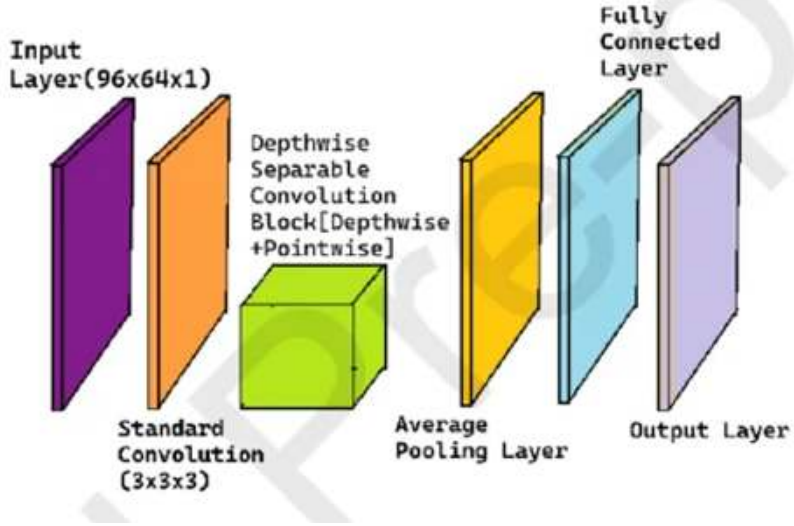


FIGURE 4.3 – Architecture de YAMNet, fondée sur des convolutions séparables en profondeur

Pour chaque fichier audio, le service `audio-preprocessing` calcule un score agrégé de parole et un score agrégé de musique à partir des classes les plus probables retournées par YAMNet, et détermine la catégorie dominante. Cette classification est stockée aux côtés des autres métadonnées du fichier et détermine son routage : seuls les fichiers classés comme parole sont transmis au service de transcription ; les fichiers classés comme musique sont conservés sans transcription ni indexation textuelle, conformément au besoin fonctionnel exprimé au chapitre 2.

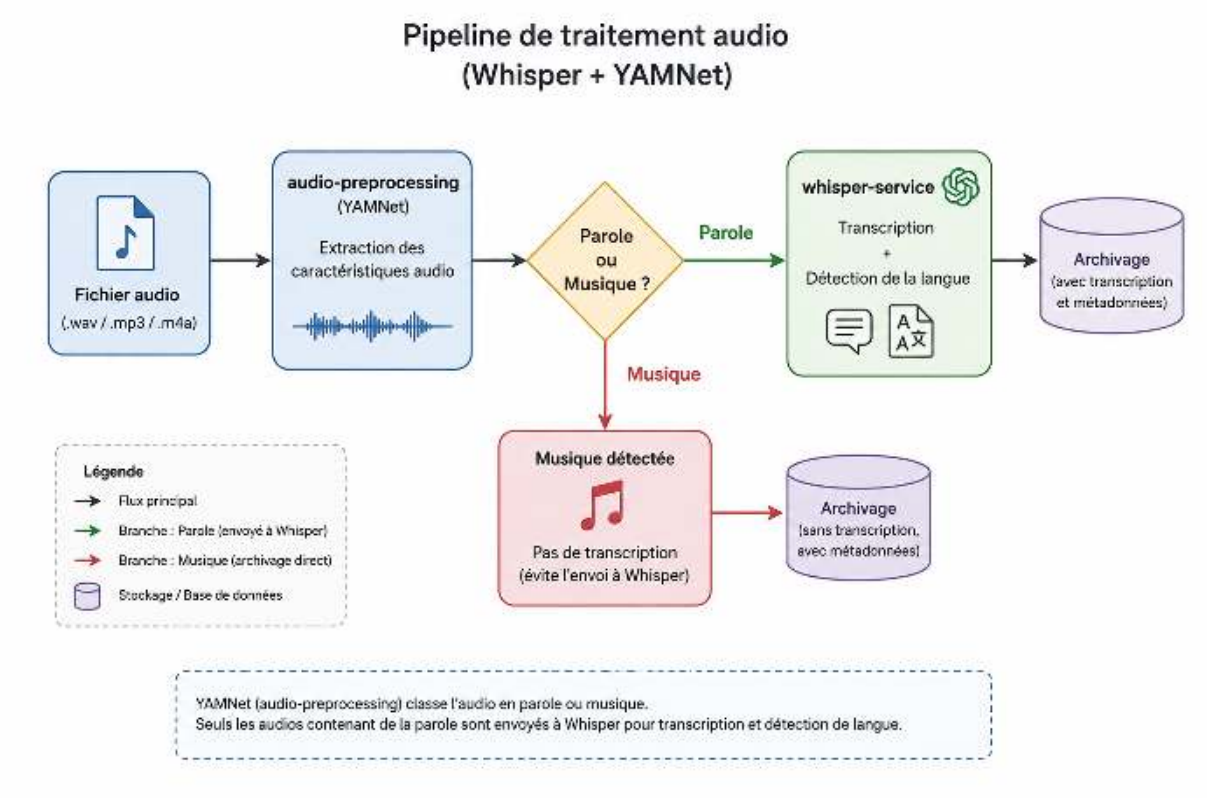


FIGURE 4.4 – Pipeline de traitement audio (flux Whisper + YAMNet)

4.3 Enrichissement par modèle de langage

Une fois le texte obtenu – qu'il provienne d'un document ou d'une transcription audio – celui-ci est soumis à un modèle de langage, exécuté localement via **Ollama**, afin de produire trois éléments d'enrichissement : un résumé synthétique, une liste de thématiques et une liste de mots-clés. Ces éléments sont indexés dans Apache Solr au même titre que le texte intégral, et permettent d'améliorer la pertinence de la recherche plein texte, en particulier lorsque la requête de l'utilisateur porte sur un sujet plutôt que sur des termes présents littéralement dans le texte.

Le choix d'un modèle de langage exécuté localement, plutôt qu'une API externe, répond directement à la contrainte de confidentialité posée au chapitre 1. Deux modèles ont été testés au cours du stage : `qwen2.5:0.5b`, très léger et rapide, utilisé pour les itérations de développement, et `llama3.2:3b`, plus capable mais plus lent à charger en mémoire, retenu pour les traitements finaux et pour l'assistant conversationnel présenté ci-après.

4.4 Assistant conversationnel : architecture RAG

4.4.1 Principe général

L'assistant conversationnel repose sur une architecture de **génération augmentée par récupération** (*Retrieval- Augmented Generation*, RAG). Ce choix répond à une limite fondamentale des modèles de langage généralistes : un tel modèle n'a aucune connaissance du contenu des archives de la SNRT, et produirait, livré à lui-même, des réponses non fondées (*hallucinations*) plutôt que de refuser de répondre. Le RAG résout ce problème en fournissant au modèle, au moment de chaque question, le contenu réellement pertinent extrait des archives, à charge pour le modèle de fonder sa réponse exclusivement sur ce contenu.

Le processus se déroule en trois temps :

1. **Récupération** (*retrieval*) – la question de l'utilisateur est transformée en représentation vectorielle (*embedding*), puis comparée aux vecteurs des fragments de documents précédemment indexés, afin d'en extraire les plus proches sémantiquement.
2. **Augmentation** du contexte – les fragments récupérés sont insérés dans une consigne (*prompt*) transmise au modèle de langage, accompagnés d'une instruction explicite lui demandant de fonder sa réponse uniquement sur ce contenu.
3. **Génération** – le modèle produit une réponse en langage naturel, accompagnée des références aux fichiers sources ayant permis de la construire.

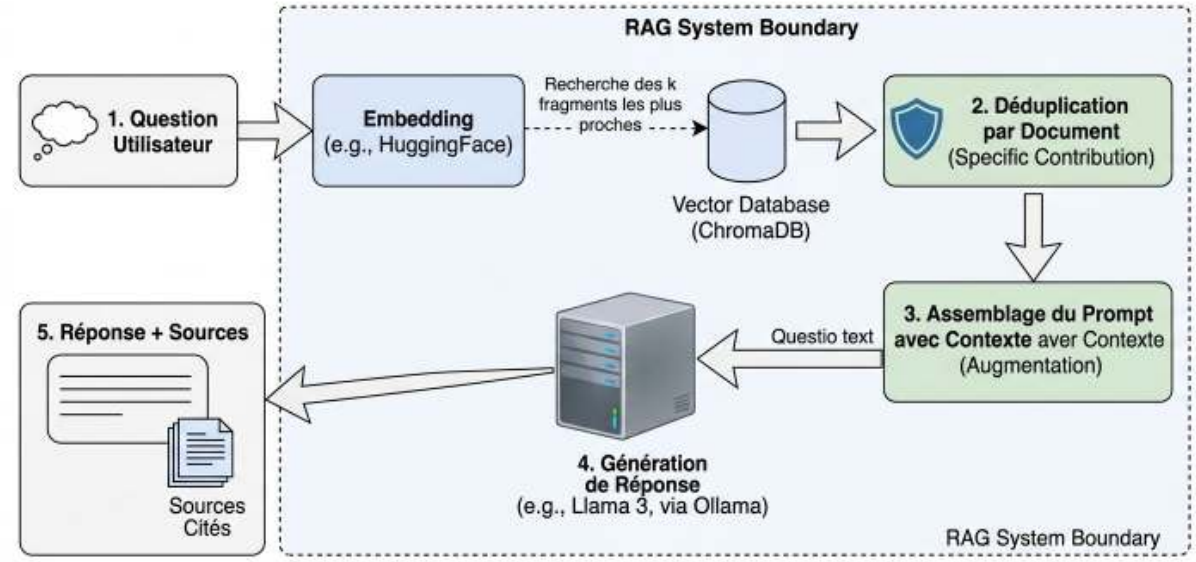


FIGURE 4.5 – Architecture RAG (retrieval → augmentation → génération)

4.4.2 Découpage des documents en fragments

Chaque document est découpé en fragments (*chunks*) avant d'être vectorisé, un document entier étant à la fois trop volumineux pour être représenté fidèlement par un unique vecteur et trop long pour tenir dans le contexte fourni au modèle de génération.

Une première version ne procédait à aucun découpage : chaque document était simplement tronqué à ses 2000 à 4000 premiers caractères avant d'être vectorisé en un seul bloc. Cette approche s'est révélée insuffisante : l'information recherchée pouvant se situer n'importe où dans le document, une troncature au début du texte la faisait purement et simplement disparaître du contenu réellement indexé, produisant des réponses de l'assistant ignorant des passages pourtant pertinents situés plus loin dans le texte.

Le découpage a été révisé sous la forme d'une fenêtre glissante à taille fixe : chaque document est découpé en fragments d'environ 800 caractères, avec un recouvrement de 100 caractères entre deux fragments consécutifs, afin de limiter le risque qu'une information pertinente ne soit coupée exactement à la frontière entre deux fragments. Chaque fragment est vectorisé et indexé indépendamment, avec une référence commune vers le document d'origine, permettant de retrouver n'importe quel passage du document, quelle que soit sa position dans le texte source.

Un ajustement complémentaire a par ailleurs été nécessaire pour les transcriptions en langue arabe : la longueur de contexte par défaut du modèle d'embedding s'est révélée insuffisante pour certains fragments, provoquant

une erreur de dépassement de contexte. Ce problème a été corrigé en augmentant explicitement la fenêtre de contexte du modèle d'embedding et en réduisant la taille maximale des fragments soumis à la vectorisation.

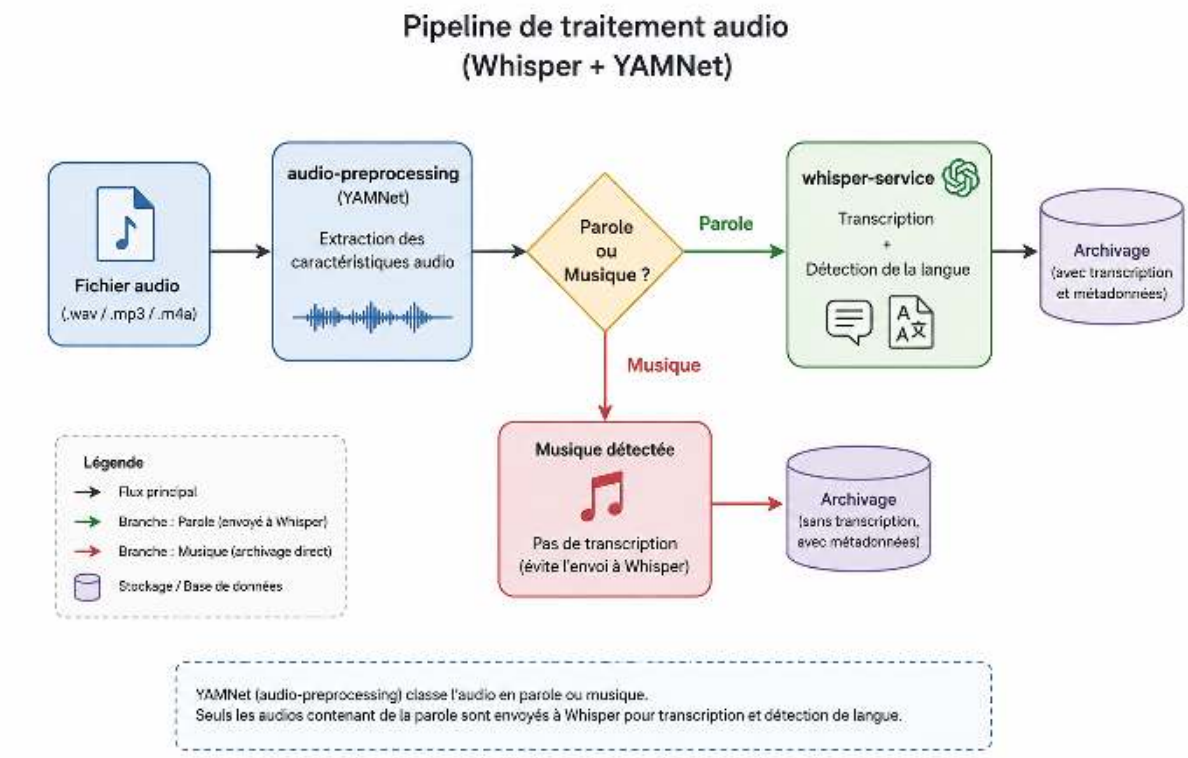


FIGURE 4.6 – Schéma du chunking à fenêtre glissante)

4.4.3 Indexation vectorielle

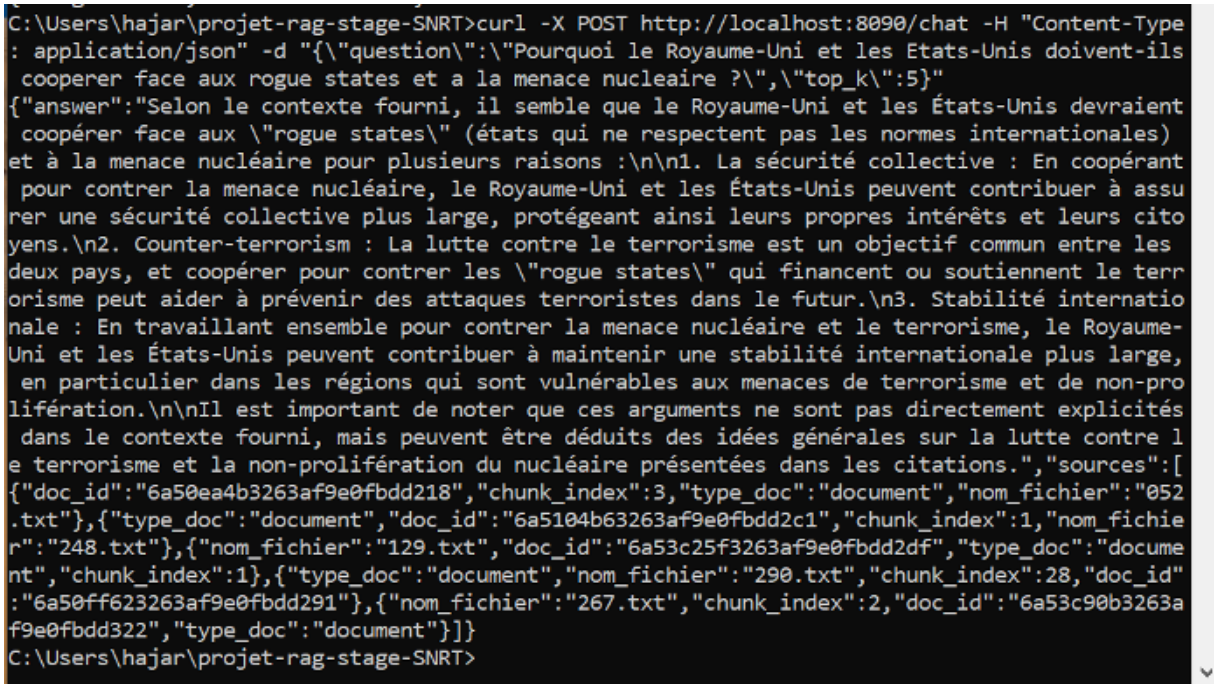
Chaque fragment est transformé en vecteur à l'aide du modèle d'embedding `nomic-embed-text`, exécuté via Ollama, puis stocké dans **ChromaDB**, avec pour métadonnées l'identifiant du document d'origine, son nom de fichier et son type (document ou audio). Cette indexation est réalisée par un script dédié, exécuté après l'indexation initiale dans MongoDB, permettant une séparation nette entre l'indexation lexicale (Solr, synchrone, réalisée par le pipeline d'ingestion) et l'indexation sémantique (ChromaDB, alimentée par ce script exécuté séparément et pouvant être reconstruite indépendamment).

4.4.4 Déduplication des sources

Le découpage d'un même document en plusieurs fragments distincts a révélé un effet de bord lors de la récupération : plusieurs des fragments les plus proches d'une question pouvaient provenir d'un seul et même document, ré-

duisant d'autant la diversité des sources réellement mobilisées pour construire la réponse. Ce comportement a été corrigé en récupérant un nombre de fragments candidats supérieur au nombre de sources finalement souhaité, puis en ne conservant qu'un seul fragment par document d'origine – le plus pertinent, les fragments étant déjà triés par similarité décroissante – jusqu'à atteindre le nombre de sources distinctes désiré.

La figure 4.7 illustre le résultat de cette correction sur une question réelle : les cinq sources retournées proviennent de cinq documents distincts (`052.txt`, `248.txt`, `129.txt`, `290.txt`, `267.txt`), alors qu'avant la mise en place de la déduplication, plusieurs fragments d'un même document apparaissaient de façon redondante parmi les sources retournées pour une même question.



```
C:\Users\hajar\projet-rag-stage-SNRT>curl -X POST http://localhost:8090/chat -H "Content-Type
: application/json" -d "{\"question\":\"Pourquoi le Royaume-Uni et les Etats-Unis doivent-ils
 cooperer face aux rogue states et a la menace nucleaire ?\",\"top_k\":5}"
{"answer":"Selon le contexte fourni, il semble que le Royaume-Uni et les États-Unis devraient
 coopérer face aux \"rogue states\" (états qui ne respectent pas les normes internationales)
et à la menace nucléaire pour plusieurs raisons :\n\n1. La sécurité collective : En coopérant
 pour contrer la menace nucléaire, le Royaume-Uni et les États-Unis peuvent contribuer à assu
rer une sécurité collective plus large, protégeant ainsi leurs propres intérêts et leurs cito
yens.\n2. Counter-terrorism : La lutte contre le terrorisme est un objectif commun entre les
deux pays, et coopérer pour contrer les \"rogue states\" qui financent ou soutiennent le terr
orisme peut aider à prévenir des attaques terroristes dans le futur.\n3. Stabilité internatio
nale : En travaillant ensemble pour contrer la menace nucléaire et le terrorisme, le Royaume-
Uni et les États-Unis peuvent contribuer à maintenir une stabilité internationale plus large,
 en particulier dans les régions qui sont vulnérables aux menaces de terrorisme et de non-pro
lifération.\n\nIl est important de noter que ces arguments ne sont pas directement explicités
 dans le contexte fourni, mais peuvent être déduits des idées générales sur la lutte contre l
e terrorisme et la non-prolifération du nucléaire présentées dans les citations.","sources":[
{"doc_id":"6a50ea4b3263af9e0fbdd218","chunk_index":3,"type_doc":"document","nom_fichier":"052
.txt"},{"type_doc":"document","doc_id":"6a5104b63263af9e0fbdd2c1","chunk_index":1,"nom_fichie
r":"248.txt"},{"nom_fichier":"129.txt","doc_id":"6a53c25f3263af9e0fbdd2df","type_doc":"docume
nt","chunk_index":1},{"type_doc":"document","nom_fichier":"290.txt","chunk_index":28,"doc_id"
:"6a50ff623263af9e0fbdd291"},{"nom_fichier":"267.txt","chunk_index":2,"doc_id":"6a53c90b3263a
f9e0fbdd322","type_doc":"document"}]}
C:\Users\hajar\projet-rag-stage-SNRT>
```


FIGURE 4.7 – Réponse de l'assistant conversationnel avec cinq sources distinctes, après correction de la déduplication

4.4.5 Génération de la réponse et prompt

Lors d'une question, cinq fragments de documents distincts sont par défaut assemblés en un contexte, inséré dans une consigne demandant explicitement au modèle de ne répondre qu'à partir de ce contexte, et d'indiquer clairement l'absence d'information plutôt que d'inventer une réponse lorsque le contexte ne permet pas de répondre. Cette contrainte, systématiquement vérifiée au cours des phases de test du projet (notamment à l'aide de questions dont la réponse n'existe pas dans le corpus), s'est révélée déterminante pour la fiabilité perçue de l'assistant.

4.4.6 Contraintes de performance

L'exécution de l'ensemble de la chaîne – transcription, embeddings, génération – sur une machine sans accélération matérielle a constitué une contrainte omniprésente durant le développement. Le paramètre `keep_alive` d'Ollama, qui contrôle la durée pendant laquelle un modèle reste chargé en mémoire après une requête, a été porté à 30 minutes, afin d'éviter qu'une question isolée ne déclenche systématiquement un rechargement complet du modèle de langage ou d'embedding.

4.5 Difficultés rencontrées et solutions

Conformément à la démarche de test systématique décrite au chapitre 1, plusieurs anomalies ont été rencontrées et documentées au cours du développement du pipeline. Les plus significatives sont résumées ci-après, avec la solution retenue dans chaque cas.

— **Dépendances manquantes au moment de l'exécution.** Certaines bibliothèques Python, requises par des fonctionnalités ajoutées en cours de projet (par exemple la validation d'adresses e-mail lors de l'inscription), n'avaient pas été déclarées dans le fichier de dépendances du service concerné, provoquant une erreur au démarrage du conteneur, révélée uniquement à l'exécution et non à la construction de l'image. *Solution :* ajout systématique de toute nouvelle bibliothèque utilisée au fichier `requirements.txt` du service correspondant, et vérification du démarrage effectif du service après chaque modification, avant de considérer une fonctionnalité comme terminée.

— **Incompatibilité de version entre bibliothèques.** Une incompatibilité est apparue entre la bibliothèque de hachage de mots de passe (`passlib`) et la version installée de sa bibliothèque de calcul sous-jacente (`bcrypt`), provoquant une erreur au moment du hachage d'un mot de passe lors de l'inscription. *Solution :* fixation explicite d'une version compatible de la bibliothèque concernée dans le fichier de dépendances, plutôt que de laisser le gestionnaire de paquets installer automatiquement la version la plus récente.

— **Routes dupliquées au sein d'un même service.** Certaines routes de l'API, modifiées à plusieurs reprises au cours du développement, ont fini par être déclarées deux fois dans le code – la seconde déclaration prenant silencieusement le pas sur la première, sans erreur explicite, source de confusion lors du diagnostic d'un comportement inattendu.

Solution : relecture systématique du fichier concerné après chaque modification importante, et suppression des déclarations obsolètes.

Au-delà de leur résolution individuelle, ces incidents ont renforcé la pertinence de la démarche de test systématique décrite au chapitre 1 : la vérification du bon fonctionnement de chaque brique de façon isolée, avant son intégration au reste du système, a permis dans la grande majorité des cas de circonscrire rapidement l'origine d'une anomalie à un service précis, plutôt que de devoir l'investiguer au sein de l'ensemble du système.

4.6 Conclusion

Ce chapitre a détaillé la chaîne de traitement automatique des données, cœur technique du projet : transcription et détection de langue des contenus audio, classification parole/musique, enrichissement par modèle de langage, et architecture de génération augmentée par récupération pour l'assistant conversationnel. Chacune de ces briques a été conçue et validée de façon incrémentale, dans le respect des contraintes de confidentialité et de ressources matérielles posées dès le cadrage du projet. La section consacrée aux difficultés rencontrées illustre par ailleurs la nature itérative du travail réalisé, chaque anomalie identifiée ayant donné lieu à une correction documentée avant la poursuite du développement. Le chapitre suivant présente la réalisation concrète de cette architecture, en particulier les interfaces utilisateur développées et les pratiques DevOps mises en place pour en assurer la fiabilité.

Chapitre 5

Réalisation

5.1 Introduction

Ce dernier chapitre présente la mise en œuvre concrète de l'architecture décrite au chapitre 3 et du pipeline détaillé au chapitre 4. Il détaille d'abord les outils et technologies mobilisés pour le développement, puis présente les principales interfaces de la plateforme telles qu'elles ont été réalisées, et enfin les pratiques DevOps mises en place pour assurer la fiabilité, la supervision et la maintenabilité de l'ensemble du système – un volet à part entière du travail réalisé durant le stage, mené en parallèle du développement fonctionnel et détaillé en section 5.3.

5.2 Outils utilisés

5.2.1 Environnement de développement

Le développement a été réalisé sur une machine locale sous Windows, à l'aide de Docker Desktop pour l'exécution et l'orchestration de l'ensemble des microservices, chacun étant testé individuellement via des requêtes `curl` avant toute intégration, conformément à la méthodologie décrite au chapitre 1.

5.2.2 Backend et bases de données

Les microservices backend sont développés en Python avec le framework **FastAPI**. La persistance des données repose sur **MongoDB** pour le stockage documentaire, **Apache Solr** pour la recherche plein texte, et **ChromaDB** pour la recherche sémantique vectorielle, ainsi que détaillé au chapitre 3.

5.2.3 Frontend

L'interface web est développée avec **React**, à l'aide de l'outil de build **Vite** pour l'environnement de développement, et packagée en production dans un conteneur servi par **nginx**.

5.2.4 Intelligence artificielle

Les traitements d'intelligence artificielle reposent sur **Whisper** (transcription et détection de langue), **YAMNet** (classification parole/musique), et **Ollama** pour l'exécution locale des modèles de langage (`llama3.2:3b`, `qwen2.5:0.5b`) et du modèle d'embedding (`nomic-embed-text`).

5.2.5 Gestion de versions

Le code source est hébergé sur un dépôt **GitHub**, organisé par microservice, chacun disposant de son propre `Dockerfile` et de son fichier de dépendances. Chaque modification a fait l'objet d'un commit dédié, décrivant précisément le changement apporté, conformément à la démarche itérative présentée au chapitre 1.

5.3 Présentation des interfaces

5.3.1 Authentification

L'accès à la plateforme nécessite la création d'un compte (adresse e-mail et mot de passe) ou la connexion à un compte existant. La figure 5.1 présente les deux écrans correspondants.

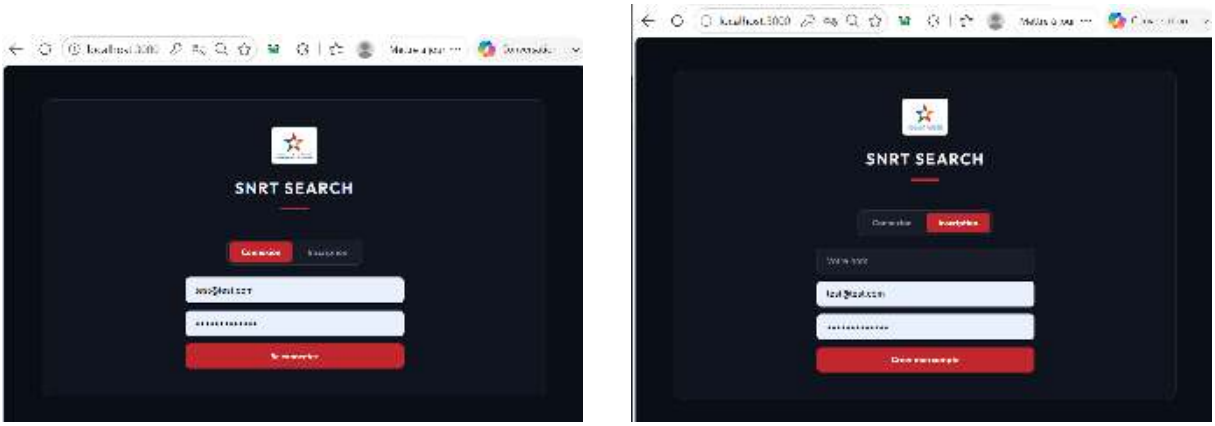


FIGURE 5.1 – Écrans de connexion et d'inscription

5.3.2 Recherche

L'écran de recherche constitue le point d'entrée principal de la plateforme. Il permet d'interroger l'ensemble du corpus indexé, avec un filtre par type de contenu, et fonctionne aussi bien sur des requêtes en français que sur des requêtes en arabe – illustrant le support multilingue exigé au chapitre 2. La figure 5.2 présente deux exemples réels : une recherche textuelle en français portant sur un document, et une recherche en arabe portant sur un fichier audio, dont le résultat affiche directement le résumé et les mots-clés générés par le pipeline décrit au chapitre 4.

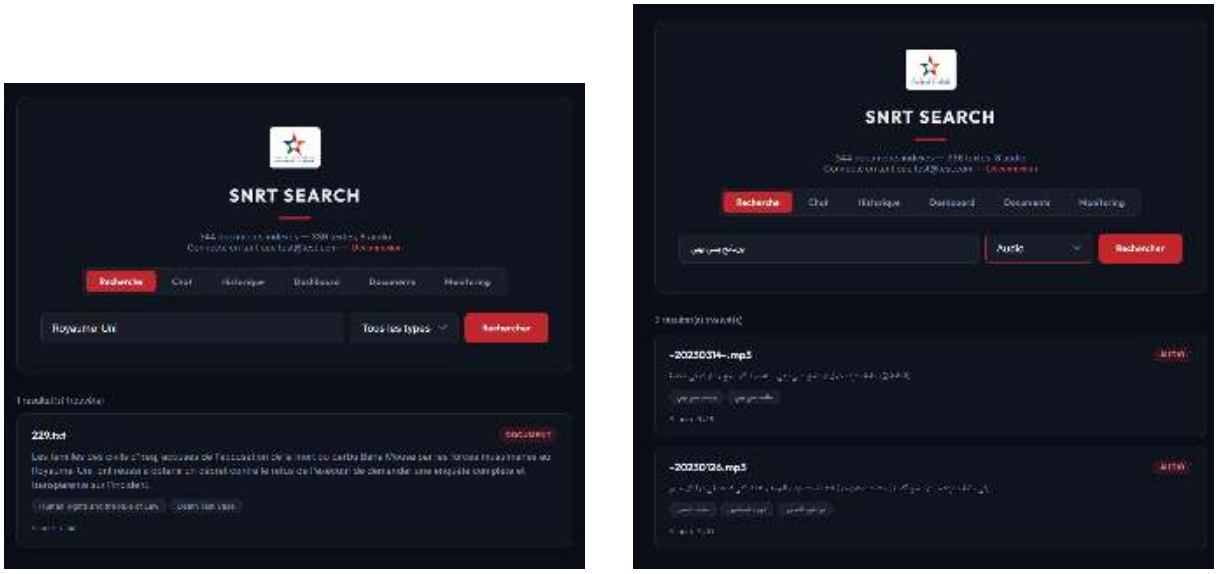


FIGURE 5.2 – Résultats de recherche sur un document français (gauche) et sur un fichier audio arabe (droite)

5.3.3 Historique

L'onglet Historique (figure 5.3) répertorie l'ensemble des recherches et des échanges conversationnels effectués par l'utilisateur, avec la possibilité de les consulter, de les relancer ou de les supprimer. Chaque entrée est horodatée et distingue visuellement une recherche d'une question posée à l'assistant conversationnel.

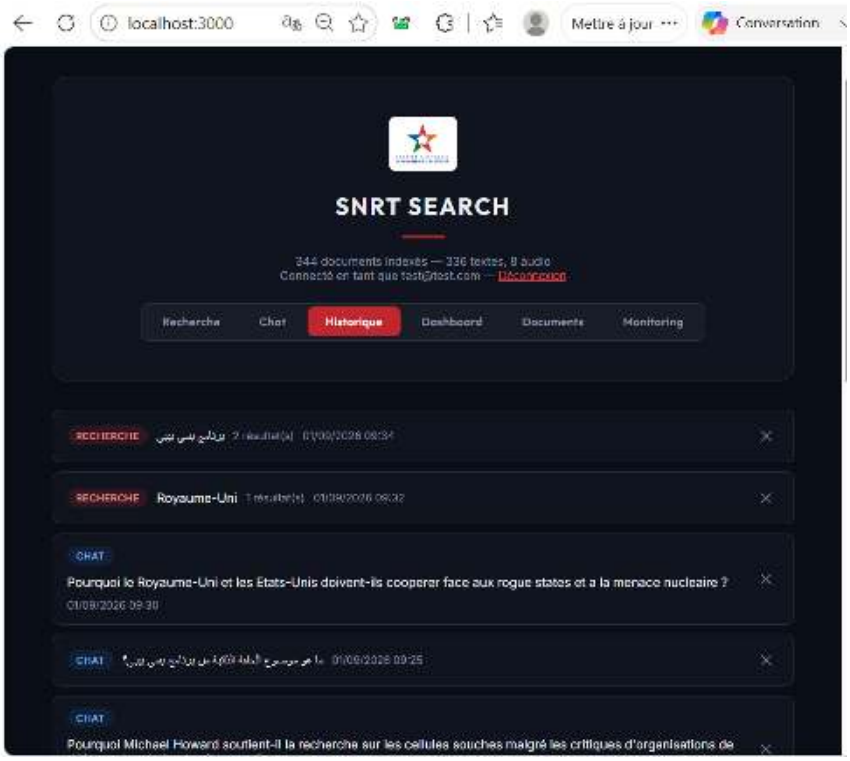


FIGURE 5.3 – Interface d'historique, mêlant recherches et conversations

5.3.4 Tableau de bord

Le tableau de bord présente, d'une part, des statistiques globales sur le corpus indexé, et d'autre part, des statistiques personnelles d'activité de l'utilisateur connecté. La figure 5.4 présente la vue d'ensemble du corpus : répartition par type de contenu (documents et fichiers audio) et par langue, ainsi que l'évolution du volume de documents ingérés dans le temps.



FIGURE 5.4 – Vue d'ensemble du corpus : répartition par type et par langue, et évolution de l'ingestion dans le temps

La figure 5.5 présente, quant à elle, la section relative à l'activité personnelle de l'utilisateur : la répartition entre recherches et conversations, et l'évolution de son activité au fil du temps.



FIGURE 5.5 – Activité personnelle de l'utilisateur : répartition recherche/chat et évolution dans le temps

5.3.5 Éditeur de documents

L'onglet Documents permet de parcourir l'ensemble des fichiers indexés, d'en consulter le contenu extrait, et de corriger manuellement ce contenu en cas d'erreur d'extraction ou d'analyse automatique. Cette interface se décline pour les deux types de contenu pris en charge par la plateforme : les documents textuels (figure 5.6) et les transcriptions audio (figure 5.7), ces dernières affichant en outre la langue détectée pour chaque fichier.

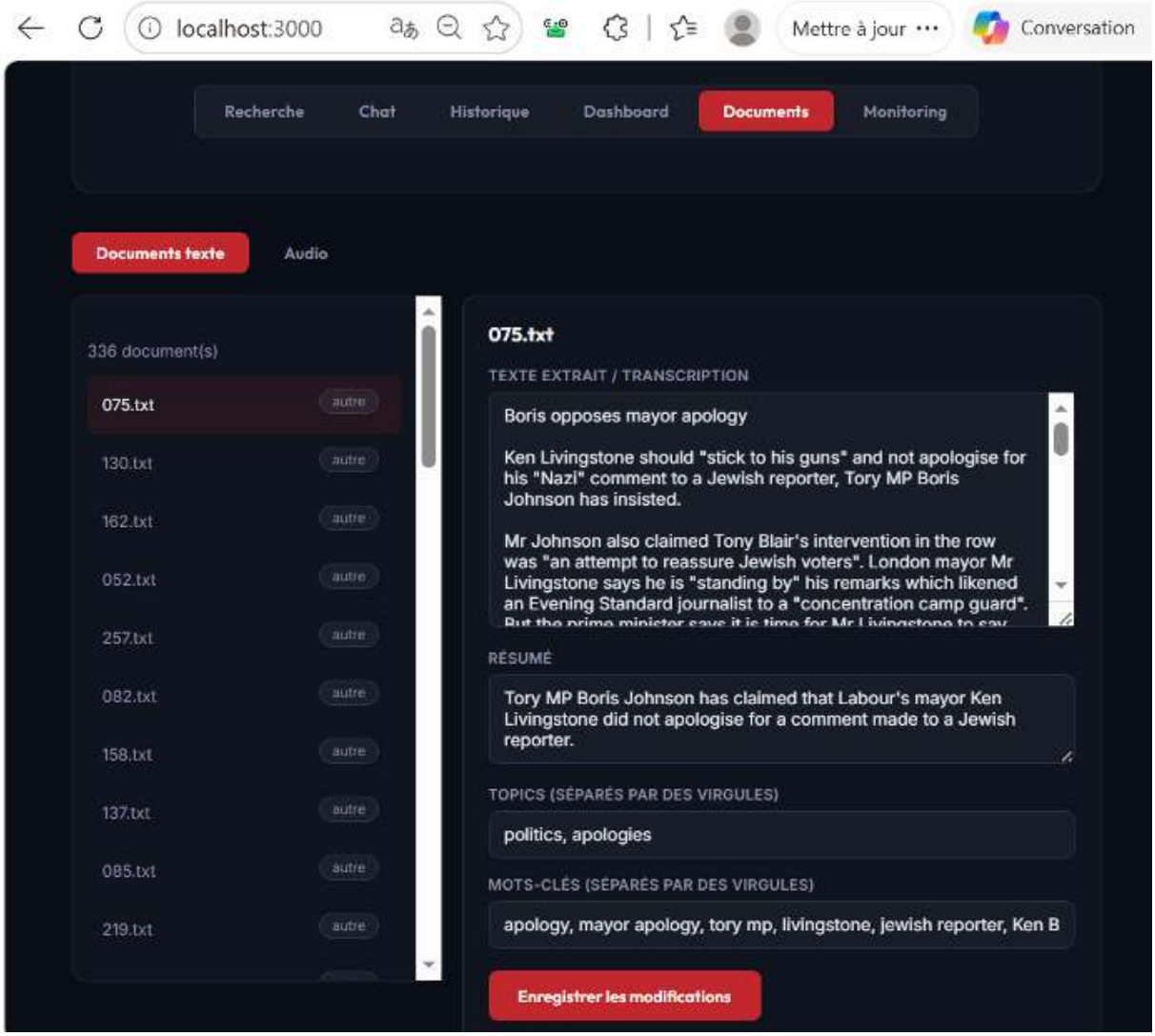


FIGURE 5.6 – Éditeur de documents : liste et vue détaillée d'un document texte

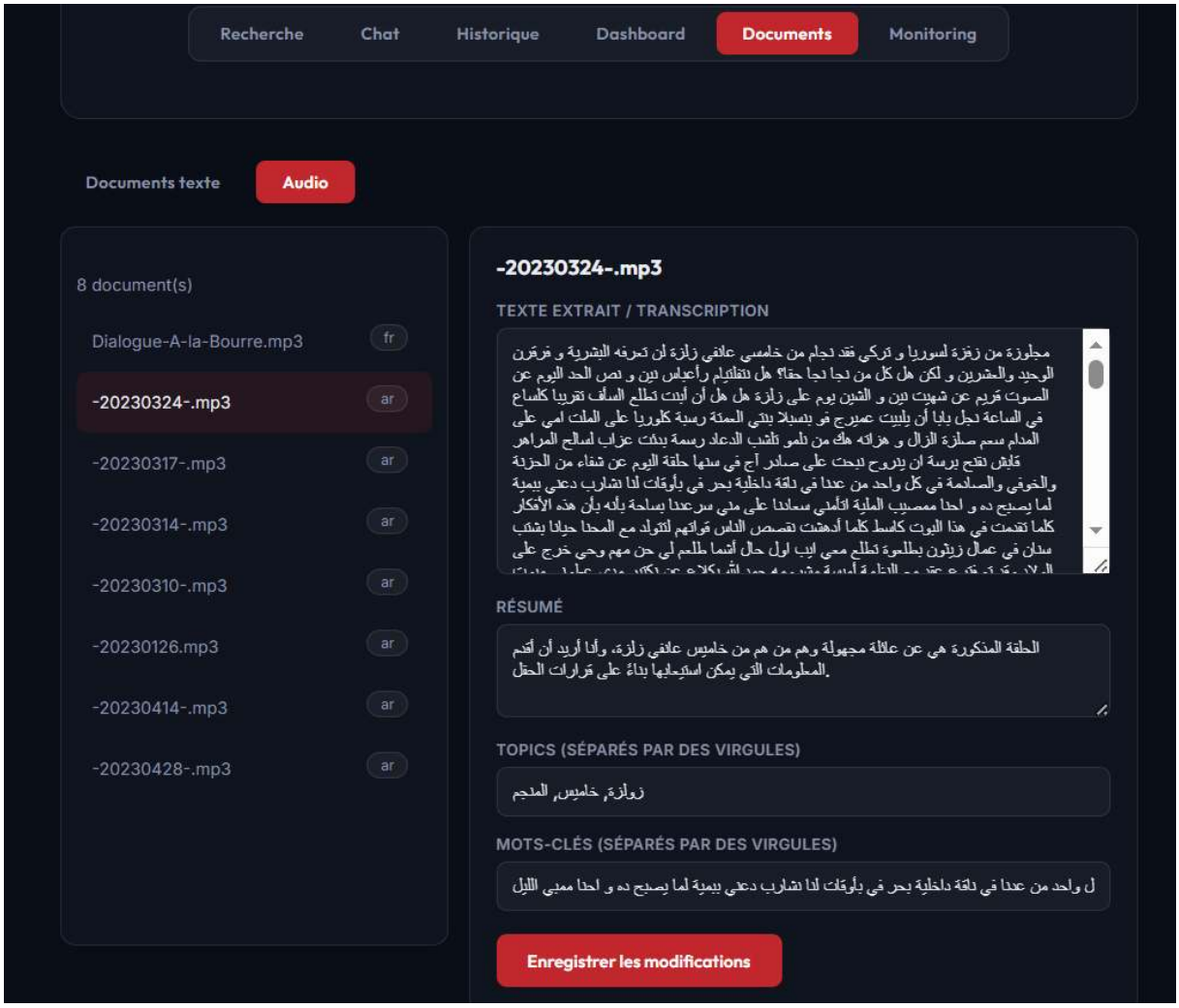


FIGURE 5.7 – Éditeur de documents : liste et vue détaillée d'une transcription audio en arabe

5.4 Pratiques DevOps

Au-delà du développement fonctionnel, une part significative du travail réalisé durant le stage a porté sur la fiabilisation et la supervision de l'ensemble du système. Cette démarche s'est déroulée en trois étapes successives, chacune ayant fait l'objet de tests et d'ajustements itératifs avant d'être considérée comme aboutie : la mise en place de contrôles de santé au niveau de l'infrastructure, la mise en place d'une intégration continue, et le développement d'un tableau de bord de supervision applicative.

5.4.1 Contrôles de santé des services

Chaque service de la plateforme, ainsi que les composants d'infrastructure partagés (MongoDB, Solr, ChromaDB, Ollama, Apache Tika), a été doté

d'un mécanisme de *healthcheck* : une vérification périodique et automatique de son bon fonctionnement, intégrée directement à la configuration Docker Compose.

La mise en place de ces contrôles n'a pas été immédiate. Une première version, fondée sur des commandes `curl` ou `wget` génériques, s'est heurtée à l'absence de ces utilitaires dans plusieurs images de base minimalistes, provoquant des services signalés comme défaillants alors même que les journaux d'exécution montraient qu'ils fonctionnaient normalement. Les vérifications ont donc été adaptées service par service : une requête HTTP native en Python pour les microservices FastAPI développés durant le stage, la commande `ollama list` pour le serveur de modèles, et l'abandon d'une vérification stricte pour les composants tiers dont l'image ne permettait pas de contrôle fiable – ces derniers restant simplement soumis à une dépendance de démarrage plutôt qu'à une condition de santé, afin de ne pas bloquer artificiellement le démarrage de l'ensemble du système. Un dernier ajustement a par ailleurs été nécessaire pour un service dont le contrôle de santé pointait vers une route inexistante, corrigée après inspection des journaux d'exécution du conteneur concerné.

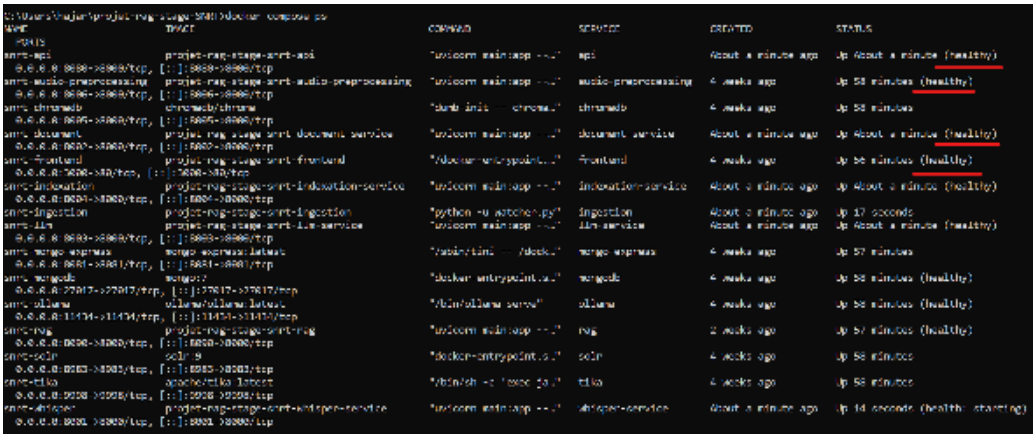


FIGURE 5.8 – État de santé de l'ensemble des services (`docker compose ps`)

5.4.2 Intégration continue

Un pipeline d'intégration continue a été mis en place à l'aide de **GitHub Actions**, se déclenchant automatiquement à chaque envoi de code vers le dépôt. Ce pipeline exécute trois vérifications :

— une vérification de la syntaxe de l'ensemble des fichiers Python du projet, rapide et permettant de détecter une erreur évidente avant tout autre traitement ;

— une vérification de la compilation du frontend React ;

— la construction, en parallèle, de l'image Docker de chacun des neuf microservices du projet, garantissant qu'aucune modification n'a rendu un `Dockerfile` ou une dépendance incompatible.

Cette intégration continue permet de détecter une régression dès l'envoi du code, avant même un déploiement local, renforçant la fiabilité du processus de développement itératif décrit au chapitre 1.

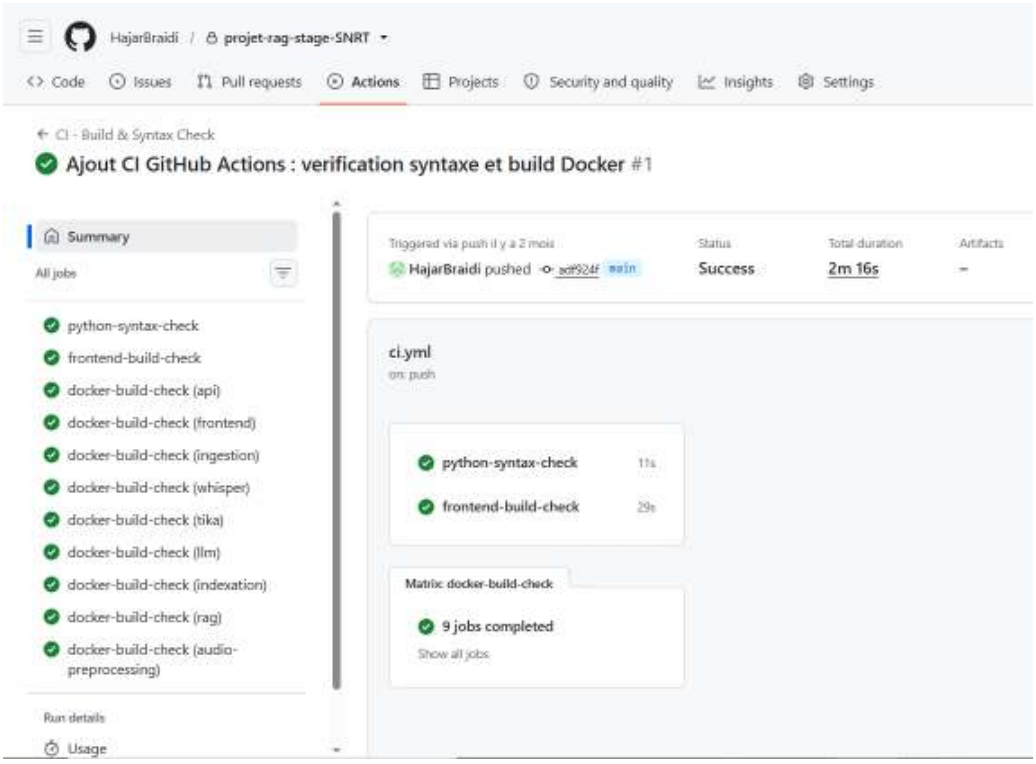


FIGURE 5.9 – Exécution réussie du pipeline d'intégration continue

5.4.3 Supervision applicative

En complément des contrôles de santé au niveau de l'infrastructure, un tableau de bord de supervision applicative a été développé au sein même de la plateforme. Celui-ci interroge en continu le point de contrôle de santé de chacun des services – y compris les composants tiers tels que MongoDB, Solr, ChromaDB et Ollama – et affiche leur état ainsi que leur temps de réponse, avec un rafraîchissement automatique toutes les dix secondes.

La mise au point de ce tableau de bord a elle-même nécessité un ajustement itératif : une première version, utilisant un délai d'attente de trois secondes par service interrogé, signalait à tort Solr et Apache Tika comme indisponibles alors qu'ils répondaient correctement, simplement avec un temps de latence légèrement supérieur sur le réseau interne Docker lors du premier appel. Ce délai a été porté à cinq secondes, résolvant le problème. La fi-

gure 5.10 présente ce tableau de bord une fois l'ensemble des douze services – y compris l'API elle-même – confirmés opérationnels.

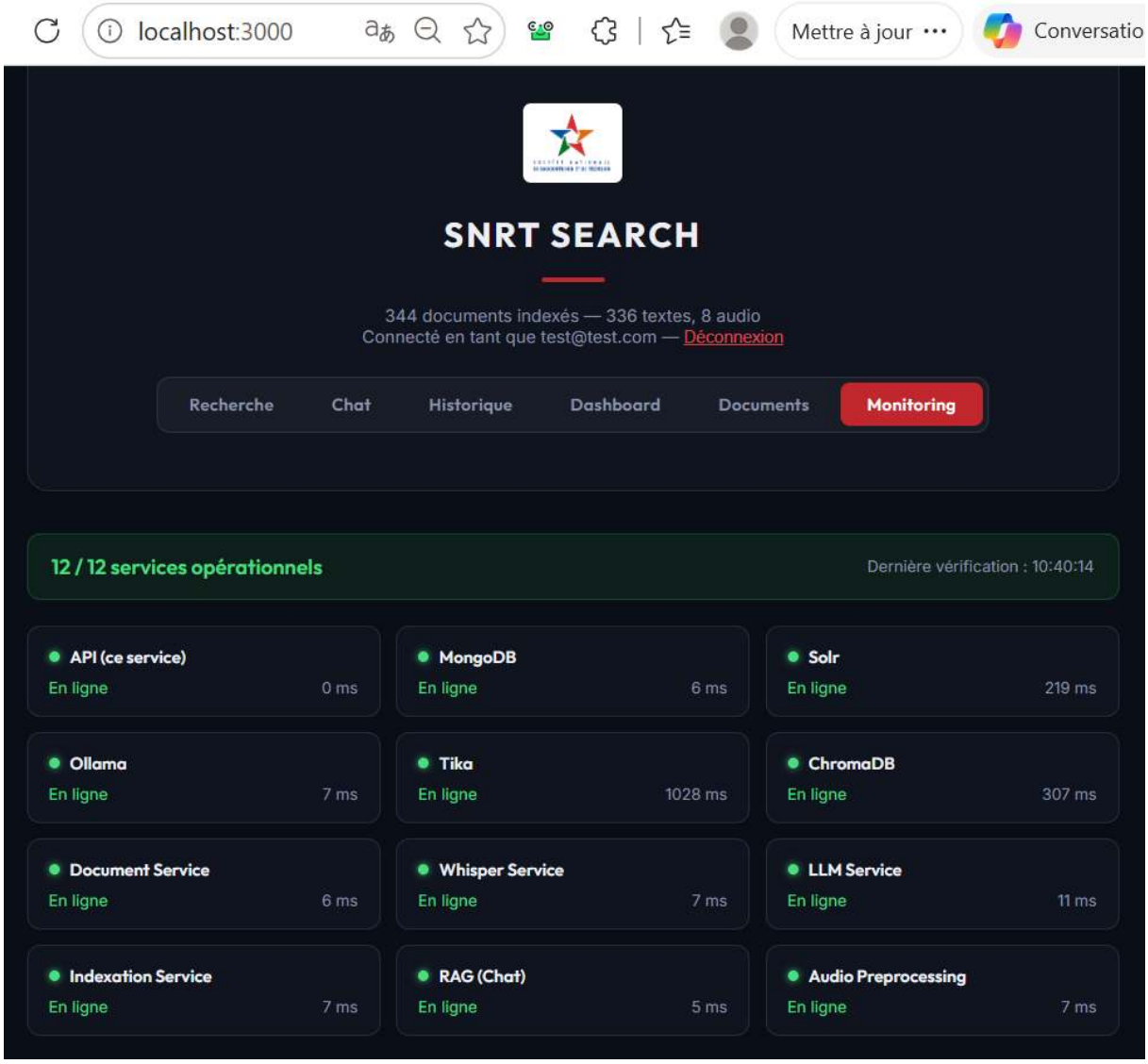


FIGURE 5.10 – Tableau de bord de supervision applicative, douze services opérationnels

Ce tableau de bord constitue un complément à la supervision au niveau infrastructure : il offre une vue consolidée et immédiatement lisible de l'état du système dans son ensemble, accessible directement depuis l'interface utilisateur, sans nécessiter d'inspecter individuellement chaque conteneur via la ligne de commande.

5.5 Conclusion

Ce chapitre a présenté la réalisation concrète de la plateforme SNRT Search : les outils et technologies mobilisés, les interfaces développées pour

chacune des fonctionnalités identifiées au chapitre 2, ainsi que les pratiques DevOps mises en place pour en assurer la fiabilité. La section consacrée à ces pratiques illustre, au même titre que la section du chapitre 4 dédiée aux difficultés rencontrées, la nature itérative du travail réalisé : chacun des trois volets DevOps présentés – contrôles de santé, intégration continue, supervision applicative – a nécessité plusieurs cycles d'ajustement avant d'atteindre son état final. Développement fonctionnel, intelligence artificielle et DevOps ont ainsi été menés de façon conjointe tout au long du stage, chacun contribuant à la robustesse et à la maintenabilité de l'ensemble du système présenté dans ce rapport.

Conclusion Générale

Ce stage, effectué au sein du département Radio, service Archivage, de la SNRT, a permis de concevoir et de développer une plateforme de recherche et d'interrogation intelligente des archives audiovisuelles et documentaires de l'établissement. L'ensemble des objectifs fixés au chapitre 1 a été atteint : mise en place d'un pipeline d'ingestion automatique traitant conjointement documents et fichiers audio, transcription et détection de langue fiables, classification automatique parole/musique, enrichissement du contenu par un modèle de langage exécuté localement, double indexation plein texte et sémantique, assistant conversationnel fondé sur une architecture RAG, interface web complète pour l'utilisateur final, et mise en place de pratiques DevOps assurant la fiabilité du système.

Sur le plan personnel, ce stage a été l'occasion de mettre en pratique, sur un projet réel et de bout en bout, des compétences relevant à la fois de l'ingénierie des données – conception d'un pipeline de traitement de données hétérogènes, choix et intégration de modèles d'intelligence artificielle sous contrainte de ressources – du développement logiciel fullstack, et des pratiques DevOps. La démarche itérative adoptée tout au long du projet, avec un test systématique de chaque fonctionnalité avant son intégration, s'est révélée déterminante pour mener à bien un projet de cette ampleur dans le temps imparti, et a permis de documenter précisément chaque difficulté rencontrée et sa résolution, comme détaillé au chapitre 4.

Plusieurs pistes d'amélioration ont été identifiées au cours du stage, que les contraintes de temps n'ont pas permis d'explorer :

— l'intégration de l'indexation vectorielle (ChromaDB) comme étape automatique du pipeline d'ingestion, au même titre que l'indexation Solr, plutôt que comme un script exécuté séparément ;
— l'utilisation d'une variante supérieure de Whisper (`base` ou `small`), sous réserve de ressources de calcul suffisantes, afin d'améliorer la qualité des transcriptions ;
— l'introduction d'un profil administrateur distinct de l'utilisateur standard, pour la supervision et la modération de la plateforme ;
— l'exploration de mécanismes d'identification du contenu musical par empreinte acoustique (*fingerprinting*), afin d'enrichir le traitement actuellement limité, pour ce type de contenu, à un simple archivage sans

indexation ;
— l'évaluation de bibliothèques dédiées à la construction de pipelines RAG (telles que LangChain ou LlamaIndex), ainsi que d'alternatives de stockage vectoriel, en remplacement de l'implémentation développée spécifiquement durant le stage.

Ces pistes, en partie issues des réflexions menées dès le cadrage du projet (voir annexe 1), constituent des prolongements naturels du travail réalisé et pourront faire l'objet de développements futurs au sein du service Archivage de la SNRT.

Bibliographie

[1] OpenAI, *Introducing Whisper*, 2022. https://openai.com/index/whisper/

[2] Google Research, *YAMNet : pretrained audio classification model*, dépôt TensorFlow Models. https://github.com/tensorflow/models/tree/master/research/audioset/yamnet

[3] FastAPI – Documentation officielle. https://fastapi.tiangolo.com/

[4] MongoDB – Documentation officielle. https://www.mongodb.com/docs/

[5] Apache Solr – Documentation officielle. https://solr.apache.org/guide/

[6] ChromaDB – Documentation officielle. https://docs.trychroma.com/

[7] Ollama – Documentation officielle. https://ollama.com/

[8] Docker – Documentation officielle. https://docs.docker.com/

[9] React – Documentation officielle. https://react.dev/

[10] Lewis, P. et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. https://arxiv.org/abs/2005.11401

Annexes

Annexe 1 – Document de travail initial : pistes d'architecture envisagées

La figure 5.11 reproduit un document de travail manuscrit établi lors des premières réflexions sur le projet. Il ne représente pas l'architecture finalement réalisée, décrite au chapitre 3, mais l'ensemble des pistes envisagées à ce stade, incluant plusieurs alternatives non retenues (identification musicale par empreinte acoustique, bibliothèques dédiées au RAG, stockage vectoriel alternatif). Ce document a servi de point de départ à la réflexion et a orienté plusieurs des perspectives d'évolution évoquées en conclusion générale de ce rapport.

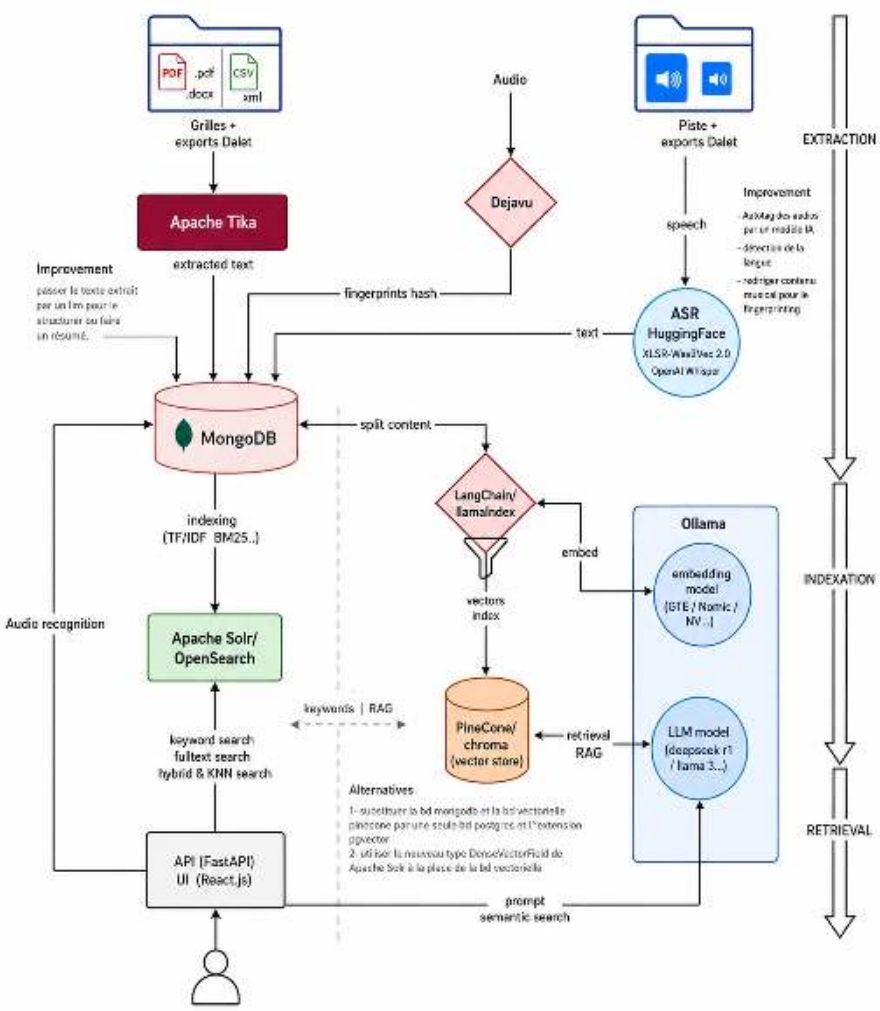


FIGURE 5.11 – Document de travail initial (pistes d'architecture envisagées, non toutes retenues)